ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

School of Engineering			
Department of Computer Science and Engineering			
Course Code:		Semester:	V
Course Title:	AI & Machine Learning	Year:	2024 - 2025
Faculty Name:	Prof. Shilpa Sudheendran		

MODULE 2

Syllabus

SOLVING PROBLEMS BY SEARCHING: Problem-Solving Agents, Example Problems

LOGICAL AGENTS: Knowledge-Based Agents, Logic, Propositional Logic: A Very Simple Logic

LEARNING FROM EXAMPLES: Forms of Learning

(Text Book 1: Chapter 3 : 3.1, 3.2, Chapter 7: 7.1, 7.3, 7.4, Chapter 19: 19.1)

ANN: Introduction, The Perceptron, Training Perceptron, Learning Boolean Functions, Multilayer Perceptron, Learning Time, Deep Learning

(Text Book 2: Chapter 11 : 11.1 – 11.5, 11.12 , 11.13

Solving Problems by Searching

Problem Solving agents

- It is a type of agent designed to find solutions to specific problems by exploring various actions or sequences of actions.
- It follows a systematic approach to solve problems that can be represented as a search through a space of possible states.
- The computational process it undertakes is called **search**.

Solving Problems by Searching

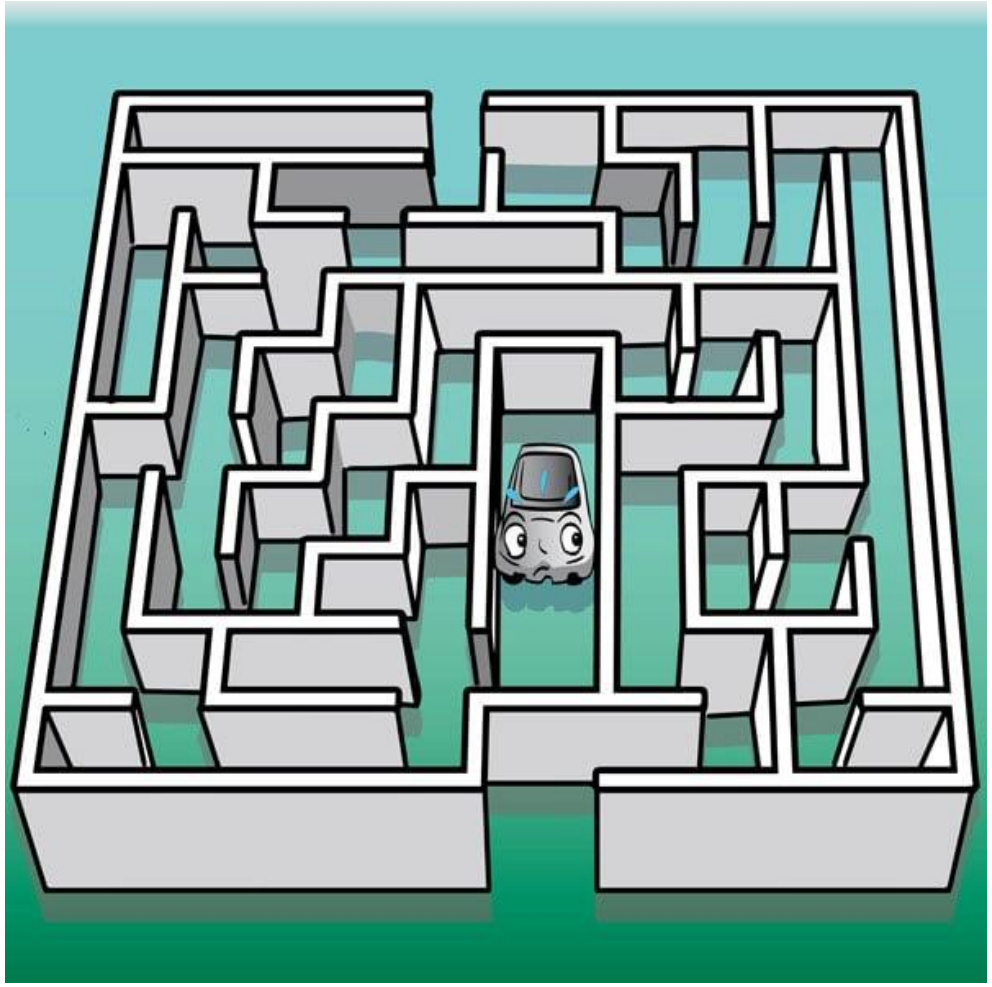
- Problem-solving agents use atomic representations.
- In atomic representations, states are treated as indivisible entities without internal structure.
- Example: An agent is placed in a 5x5 grid world. The agent needs to reach a goal located at a specific grid cell while avoiding obstacles

Problem Solving Agent

- **Key Concepts of Problem-Solving Agents:**
  - **Goal**: The desired state the agent aims to achieve.
  - **Initial State**: The starting point or current condition from which the agent begins.
  - **Actions**: The possible operations the agent can take to move from one state to another.
  - **Transition Model**: A model that predicts the result of an action taken in a state.
  - **State Space**: The set of all possible states that can be reached by the agent.
  - **Solution**: A sequence of actions leading from the initial state to the goal state.

Problem Solving Agent

- Example : Maze – Solving Robot



Planning Agents

- It is a type of problem-solving agent that determines a sequence of actions (a plan) to achieve a specific goal, often by reasoning about the future.
- **Reactive agents**, which decide on actions based only on the current state,
- **Planning agents** simulate and evaluate future states to find optimal or feasible sequences of actions.

Planning Agents

- Agents that use factored or structured representations of states are called planning agents.
- Example: A robot needs to navigate through a grid to reach a charging station. The grid has obstacles that the robot must avoid, and it starts at a specific location.

Planning Agent

- Example: Delivery Drone

Informed and Uninformed Algorithms

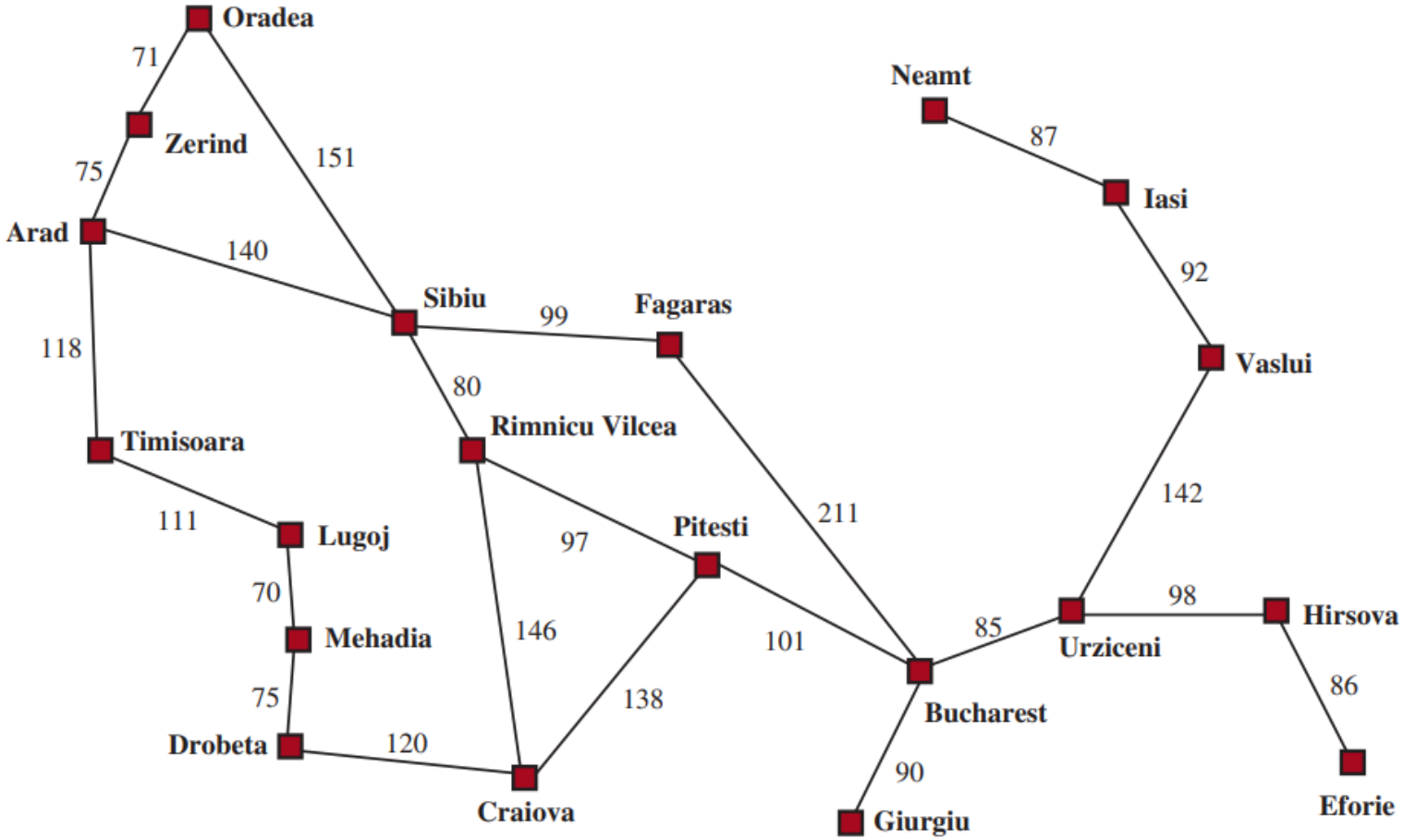
- Informed algorithms, in which the agent can estimate how far it is from the goal
- Uninformed algorithms, where no such estimate is available.

Informed and Uninformed Algorithms

- Example: Imagine an agent enjoying a touring vacation in Romania.
- Suppose the agent is currently in the city of Arad and has a nonrefundable ticket to fly out of Bucharest the following day.
- The agent observe street signs and sees that there are three roads leading out of Arad: one toward Sibiu, one to Timisoara, and one to Zerind.
- None of these are the goal, so unless the agent is familiar with the geography of Romania, it will not know which road to follow.
- If the agent has no additional information—that is, if the environment is unknown—then the agent can do no better than to execute one of the actions at random.

Example

A simplified road map of part of Romania, with road distances in miles.



Example

- With that information, the agent can follow this **four-phase problem-solving process:**
- **Goal formulation:** The agent adopts the goal of reaching Bucharest. Goals organize behavior by limiting the objectives and hence the actions to be considered.
- **Problem formulation:** The agent devises a description of the Problem formulation states and actions necessary to reach the goal—an abstract model of the relevant part of the world.
- For our agent, one good model is to consider the actions of traveling from one city to an adjacent city

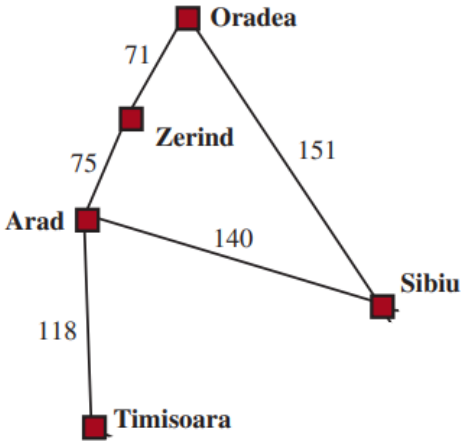
Example

- **Search:** Before taking any action in the real world, the agent simulates sequences of actions in its model, searching until it finds a sequence of actions that reaches the goal.It's about exploring different options to discover a viable solution.
  - It might try several paths, such as Arad → Sibiu → Fagaras → Bucharest, and check if they lead to the goal.
- **Solution:**A solution is a sequence of actions that successfully leads the agent from the initial state to the goal.
  - In the example going from Arad to Sibiu to Fagaras to Bucharest.
- **Execution:** The agent can now execute the actions in the solution, one at a time.

Search Problem:A search problem can be defined formally as follows:

1. A set of all possible states that the environment can be in is called as **state space**.
2. The **initial state** that the agent starts in. For example: Arad.
3. A set of one or more **goal states**. Sometimes there is one goal state (e.g., Bucharest).There may be different goal states also.
4. The actions available to the agent. An example:

   ACTIONS(Arad) = {ToSibiu,ToTimisoara,ToZerind}.



Cont..Search Problem

5.**A transition model**, which describes what each action does.

RESULT(Arad, ToZerind) = Zerind

6. An **action cost function**, denoted by ACTION-COST(s,a, s′) when we are programming c(s,a, s′) when we are doing math, that gives the numeric cost of applying action a in state s to reach state s′.

For route-finding agents, the cost of an action might be the length in miles.

Cont..Search Problem

Path Example:

- Path: Arad → Sibiu → Fagaras → Bucharest

Optimal Solution:

- **Find the path with the minimum total distance, for example, Arad → Sibiu → Fagaras → Bucharest, which might have the least total distance compared to other possible paths.**

Graph i The state space can be represented as **a graph i**n which the vertices are states and the Graph directed edges between them are actions.

Standardized Problems

- These problems are designed to illustrate or test different **problem-solving methods** or algorithms. They serve as **benchmarks** to help researchers or developers compare how well different algorithms perform.

- Concise and Exact: A standardized problem can be described in a **clear and specific** way, making it easy to communicate the problem and share it with others. The rules, conditions, and goals are well-defined and do not change.

Standardized Problems

- Researchers use standardized problems to test and evaluate algorithms under identical conditions. This makes it possible to compare the performance of different algorithms fairly.
- For example, the 8-puzzle (where the goal is to arrange numbered tiles in a particular order) is a standardized problem. It has a clear description, fixed rules, and everyone can solve the exact same version of the problem.

Real World Problems

- These problems come from real-life situations that require practical solutions. Unlike standardized problems, they often involve unique and unpredictable challenges.

- Not Standardized: Real-world problems are idiosyncratic, meaning they are unique to each situation. The way they are defined or formulated depends on many specific factors, such as the environment or tools being used.

Real World Problems

- In real life, problems are not always neat and predictable. Different robots or devices might use different kinds of sensors, each providing different data. This makes it hard to create a "one-size-fits-all" solution.
- For example, a robot navigating a room might have to account for different kinds of obstacles, surfaces, or sensor data that vary depending on the robot and its environment. What works for one robot might not work for another.

Examples of problem solving

1. Grid World (fig in next slide)

- **Description:** A two-dimensional array where agents move between cells.
- **States:** Each state represents a specific configuration of the grid, including the positions of the agent and any objects like dirt. For example, in a simple two-cell vacuum world, states are defined by the positions of the agent and whether the cells contain dirt.
- **Initial State:** Can be any configuration of the grid cells.
- **Actions:** Typically include moving in four directions (Up, Down, Left, Right) or performing tasks like "Suck" in the vacuum world.
- **Transition Model:** Moves or actions change the state based on the current position and action taken. For instance, moving right from a cell updates the agent's position accordingly.
- **Goal State:** The state where all cells are clean (for the vacuum world).
- **Action Cost:** Each action has a fixed cost, often 1.

In general, a vacuum environment with n cells has $n \cdot 2^n$ states.

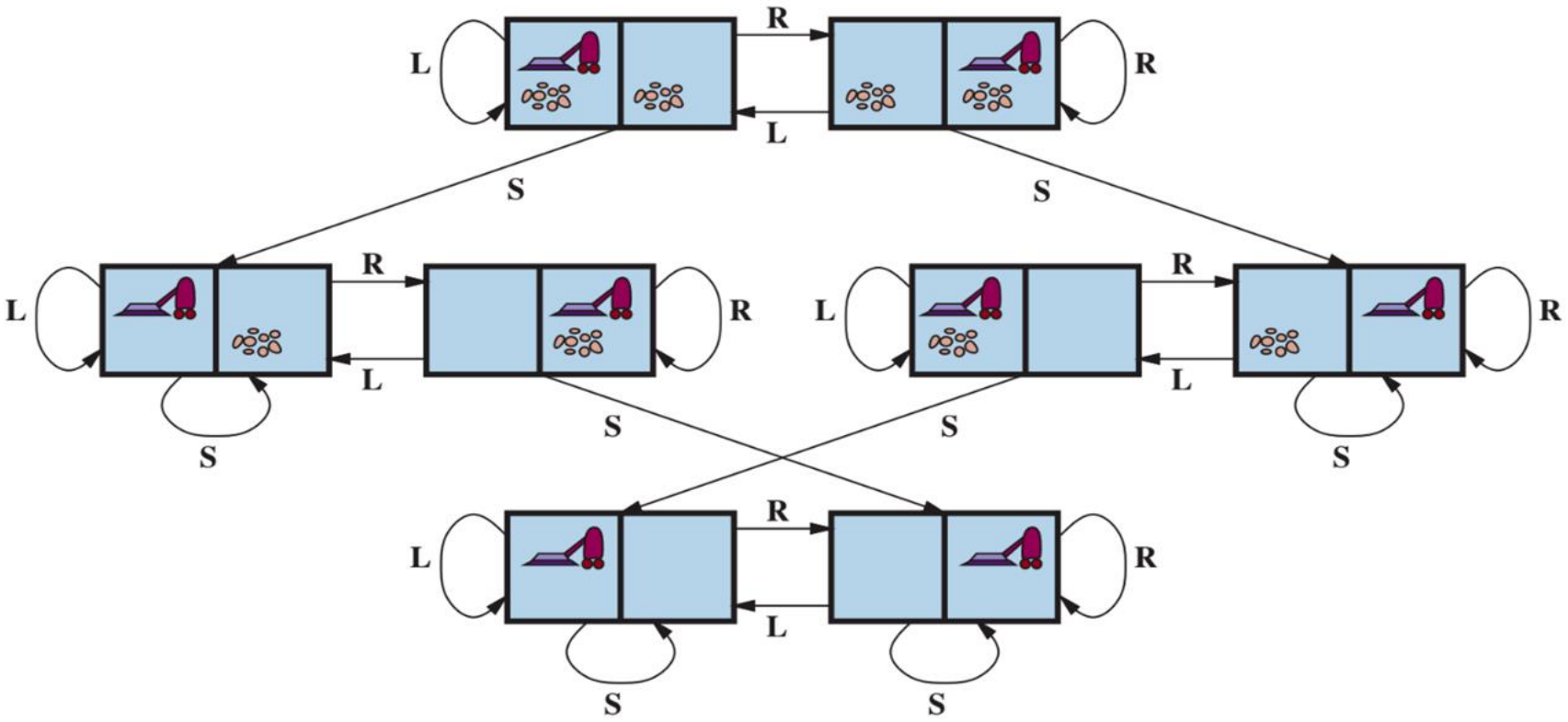


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

2. Sliding-Tile Puzzles (e.g., 8-Puzzle, try problem formulation for 15-Puzzle)

The agent can't push a box into another box or a wall. For a world with n non-obstacle cells and b boxes, there are n× n! / (b!(n− b)!) states; for example on an 8× 8 grid with a dozen boxes, there are over **200 trillion states.**

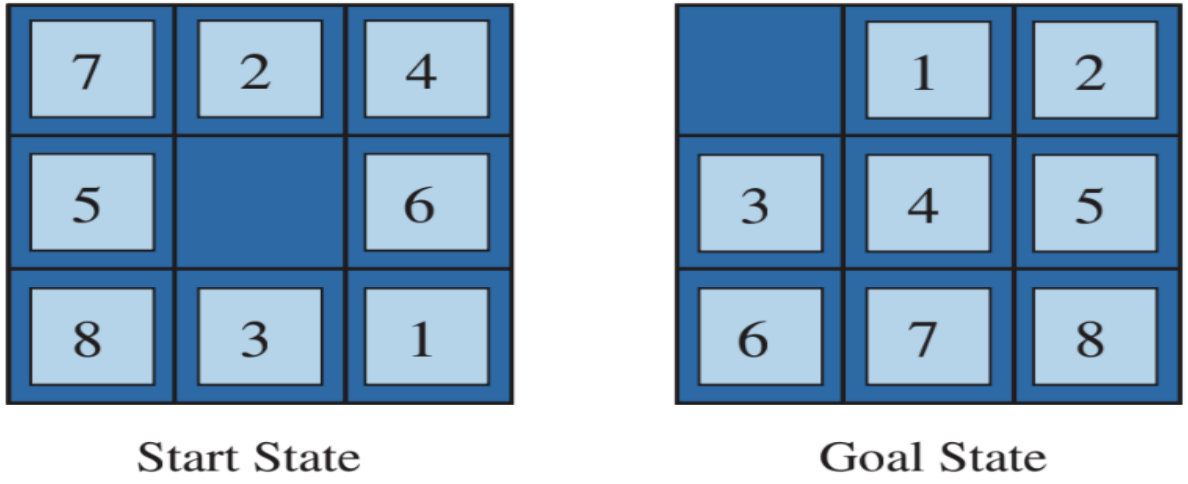


Figure 3.3 A typical instance of the 8-puzzle.

States: A state description specifies the location of each of the tiles.

Initial state: Any state can be designated as the initial state.

Actions: While in the physical world it is a tile that slides, the simplest way of describing an action is to think of the blank space moving Left, Right, Up, or Down. If the blank is at an edge or corner then not all actions will be applicable.

Transition model: Maps a state and action to a resulting state; for example, if we apply Left to the start state in Figure 3.3, the resulting state has the 5 and the blank switched.

Goal state: Although any state could be the goal, we typically specify a state with the numbers in order.

Action cost: Each action costs 1.

Try writing problem formulation for 4 and 8 queen problem

- 8 queen

- States : any arrangement of 0 to 8 Queens on the board.
- Initial State : Empty board
- Successor function : Add a Queen to an empty space
- Goal Test : 8 queens are on the board but none of them should attack each other.
- Path cost : for each action cost will be 1. so default is 8 (but the cost will be varied based on number of iterations to takes to complete the problem)

Real World Problems

- Route-finding problem
- Touring problems - Traveling salesperson problem (TSP)
- VLSI layout
- Robot navigation
- Automatic assembly sequencing - protein design

Logical Agents

- **Logical agents** are a type of artificial intelligence (AI) that use formal logic to make decisions and solve problems.
- These agents rely on a structured approach to reasoning, often involving propositional logic or first-order logic.
- It is also Known as **Knowledge Based Agents**.
- The central component of a knowledge-based agent is its **knowledge base**, or KB.

Logical Agents

- A knowledge base is a set of sentences.
- Each sentence is expressed in a language called a knowledge representation language and represents some assertion about the world.
- When the sentence is taken as being given without being derived from other sentences, we call it an **axiom**.
- **Inference—**deriving new sentences from old.

```
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
              t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Figure 7.1 A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

knowledge level and Implementation level

Knowledge level—->where we need specify only what the agent knows and what its goals are, in order to determine its behavior.

- How to modify the behavior?
  - Done at the **knowledge level**.
  - Tell the agent it's goals, and...
  - Tell the agent what it knows.
- Example:
  - Tell taxi about Bay Bridge.
  - Tell taxi to go from SF to Oakland.
  - Taxi knows to cross the bridge.
- At the **implementation level**, can be done in lots of ways.

- How to build it?
  - TELL it what it needs to know.
  - Declarative approach:
    - TELL sentences one-by-one until the agent knows what to do in its environment.
  - Procedural approach:
    - Encode the behaviors directly into computer code.
  - Best agents combine both.

Logic

- Knowledge bases consist of sentences.
- These sentences are expressed according to the **syntax** of the representation language.
- A **logic** must also define the semantics, or meaning, of sentences.
- The **semantics** defines the truth of each sentence with respect to each possible world.
- We use the term **model** in place of “possible world.” whereas possible worlds might be thought of as real environments

Logic

- **Models** are mathematical abstractions, each of which has a fixed truth value (true or false) for every relevant sentence.
- Example: If a sentence α is true in model m, we say that m satisfies α or sometimes m is a model of α.
- We use the **notation M(α)** to mean the set of all models of α.

Logical Entailment

- The idea that a sentence follows logically from another sentence is called as **logical entailment** between sentences
- A ⊨ β mean that the sentence α entails the sentence β.

- **Example:**
  - Suppose **α** is the statement "It is raining."
  - **β** could be the statement "The ground is wet."
  - If it is logically true that whenever it rains, the ground becomes wet, then we can say **α ⊨ β** (rain logically entails a wet ground).

Logic

- **Model checking** enumerates all possible models to check that α is true in all models in which KB is true, that is,

$$M(KB) \subseteq M(\alpha).$$

- An inference algorithm that derives only entailed sentences is called **sound or truth preserving.**
- Soundness is a highly desirable property
- The property of **completeness** is also desirable: an inference algorithm is complete if it can derive any sentence that is entailed.

Logic

- **Grounding**—the connection between logical reasoning processes and the real environment in which the agent exists.
- Example: “Robotic Vacuum Cleaner”
  - **Perception**: The vacuum detects dirt through its sensors. This is **grounding**—the logical process of detecting dirt is connected to a real-world measurement (sensor data).
  - **Action**: The vacuum moves to clean the detected area. The reasoning process ("if dirt is present, clean") is **grounded** in the vacuum's actual movement and cleaning behavior.
  - **Obstacle Avoidance**:
  - **Logical rule**: "If an obstacle is detected, the vacuum should turn and avoid it."
  - **Grounding**: The sensor detects a real obstacle (a chair leg), and the vacuum physically avoids it based on its logical rule.
- The vacuum cleaner's **logical reasoning** ("if dirt, then clean") must be connected to **real-world actions** and **sensory inputs** for it to function correctly in its environment. **Grounding** ensures that the agent’s reasoning isn't just abstract but is based on real-world perceptions (sensors) and results in meaningful actions (cleaning, avoiding obstacles).

Logic

- **Learning** - the process by which an agent or system acquires **general rules** or knowledge from **experience**.
- This is a crucial aspect of how intelligent agents improve over time by not just responding to individual experiences but by forming broader conclusions that can be applied to new situations.
- **Example:**
- **Perceptual Experience**: A self-driving car encounters several stop signs in different locations.
- **General Rule (Learning)**: After repeated experiences, the car "learns" a rule: "Whenever a stop sign is detected, the car must stop."
  - This rule is a **generalization** based on the car's multiple encounters with stop signs. It's not just a single memory of a specific stop sign; it's a broad, learned behavior that the car applies in any future instance when a stop sign is detected.

Propositional Logic

- **Propositional Logic**, also known as **Boolean logic**, is a formal system in logic used to represent and reason about propositions.
- It is a **simple form of logic** where statements (called **propositions**) are either **true** or **false**, and they are connected using logical operators.
- Propositional logic is widely used in areas like mathematics, computer science, and artificial intelligence for reasoning and problem-solving.

Propositional Logic

- **Propositions**:
  - A **proposition** is a declarative statement that can either be **true** or **false** but not both.
  - Examples:
    - "The sky is blue." (True or False)
    - "2 + 2 = 4." (True)
    - "It is raining." (True or False, depending on the situation)

Propositional Logic

- **Logical Connectives**: Propositional logic uses **connectives** to combine or modify propositions.
  - **Negation (¬)**: The opposite of a proposition.

    If P is true, then ¬P ("not P") is false.

    Example: If P = "It is raining," then ¬P = "It is not raining."
  - **Conjunction (∧)**: A logical **AND** between two propositions.

    P∧Q is **true** if **both** P and Q are true.

    Example: "It is raining AND it is cold."

Propositional Logic

- **Logical Connectives**:
  - **Disjunction (∨)**: A logical **OR** between two propositions.

    P∨Q is **true** if **at least one** of P or Q is true.

    Example: "It is raining OR it is sunny."
  - **Implication (→)**: A logical **IF-THEN** relationship.

    P→Q means "if P, then Q." It is **false** only when P is true and Q is false.

    Example: "If it rains, then the ground will be wet."
  - **Biconditional (↔)**: A logical **IF AND ONLY IF** relationship.

    P↔Q means "P is true if and only if Q is true." It is true when both propositions have the same truth value.

    Example: "You can enter if and only if you have a ticket."

Propositional Logic

- A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

$$
\begin{array}{rcl}
Sentence & \rightarrow & AtomicSentence \mid ComplexSentence \\
AtomicSentence & \rightarrow & True \mid False \mid P \mid Q \mid R \mid \ldots \\
ComplexSentence & \rightarrow & (\, Sentence \,) \\
 & \mid & \neg\, Sentence \\
 & \mid & Sentence \wedge Sentence \\
 & \mid & Sentence \vee Sentence \\
 & \mid & Sentence \Rightarrow Sentence \\
 & \mid & Sentence \Leftrightarrow Sentence
\end{array}
$$

$$
\text{OPERATOR PRECEDENCE} \quad : \quad \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
$$

Propositional Logic

- **Truth Tables**: Truth tables show how the truth values of complex propositions are derived from the truth values of their components.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Propositional Logic

- **Tautology and Contradiction**:
  - A **tautology** is a proposition that is always true, regardless of the truth values of the individual propositions.
    - Example: $P \vee \neg P$ (This is always true because either P is true or its negation is true.)
  - A **contradiction** is a proposition that is always false.
    - Example: $P \wedge \neg P$ (This is always false because P cannot be true and false at the same time.)

Propositional Logic: A very Simple Logic

Syntax

- The syntax of propositional logic defines the allowable sentences.
- The atomic sentences: Atomic sentences consist of a single proposition symbol.
- Each such symbol stands for a proposition that can Proposition symbol be true or false.
- We use symbols that start with an uppercase letter and may contain other letters or subscripts, for example: P, Q, R, $W_{1,3}$ and FacingEast
- There are two proposition symbols with fixed meanings: True is the always-true proposition and False is the always-false proposition.

Propositional Logic: A very Simple Logic

- **Semantics**
- The semantics defines the rules for determining the truth of a sentence with respect to a particular model.
- In propositional logic, a model simply sets the truth value—true or false—for every proposition symbol.
- For example, if the sentences in the knowledge base make use of the proposition symbols $P_{1,2}$, $P_{2,2}$, and $P_{3,1}$, then one possible model is

$$m1 = \{P_{1,2}=\text{false}, P_{2,2}=\text{false}, P_{3,1}=\text{true}\}.$$

- With three proposition symbols, there are $2^3=8$ possible models.

Propositional Logic: A very Simple Logic

- The semantics for propositional logic must specify how to compute the truth value of any sentence, given a model.
- All sentences are constructed from atomic sentences and the five connectives
- Atomic sentences are easy:
- True is true in every model and False is false in every model.
- The truth value of every other proposition symbol must be specified directly in the model. For example, in the model m1 given earlier, $P_{1,2}$ is false.

Learning from Example

- An agent is learning if it improves its performance after making observations about the world.
- When the agent is a computer, we call it **Machine Learning**: a computer observes some data, builds a model based on the Machine learning data, and uses the model as both a hypothesis about the world and a piece of software that can solve problems.

Forms of Learning

- Any component of an agent program can be improved by machine learning.
- The improvements,and the techniques used to make them, depend on these factors:

✓ Which component is to be improved.

✓ What prior knowledge the agent has, which influences the model it builds.

✓ What data and feedback on that data is available.

Forms of Learning

- The components of these agents include:

1. A direct mapping from conditions on the current state to actions.

2. A means to infer relevant properties of the world from the percept sequence.

3. Information about the way the world evolves and about the results of possible actions the agent can take.

4. Utility information indicating the desirability of world states.

5. Action-value information indicating the desirability of actions.

Forms of Learning

6. Goals that describe the most desirable states.

7. A problem generator, critic, and learning element that enable the system to improve.

- Consider a self-driving car agent that learns by observing a human driver.
- Every time the driver brakes, the agent might learn a condition–action rule for when to brake (component 1).
- By seeing many camera images that it is told contain buses, it can learn to recognize them (component 2).
- By trying actions and observing the results—for example, braking hard on

Forms of Learning

a wet road—it can learn the effects of its actions (component 3).

- Then, when it receives complaints from passengers who have been thoroughly shaken up during the trip, it can learn a useful component of its overall utility function (component 4).

Forms of Learning

- In **supervised learning** the agent observes input-output pairs and learns a function that maps from input to output.
- For example, the inputs could be camera images, each one accompanied by an output saying “bus” or “pedestrian,” etc.
- An output like this is called a label. T
- In **unsupervised learning** the agent learns patterns in the input without any explicit feedback.
- The most common unsupervised learning task is clustering: detecting potentially useful clusters of input examples.

Forms of Learning

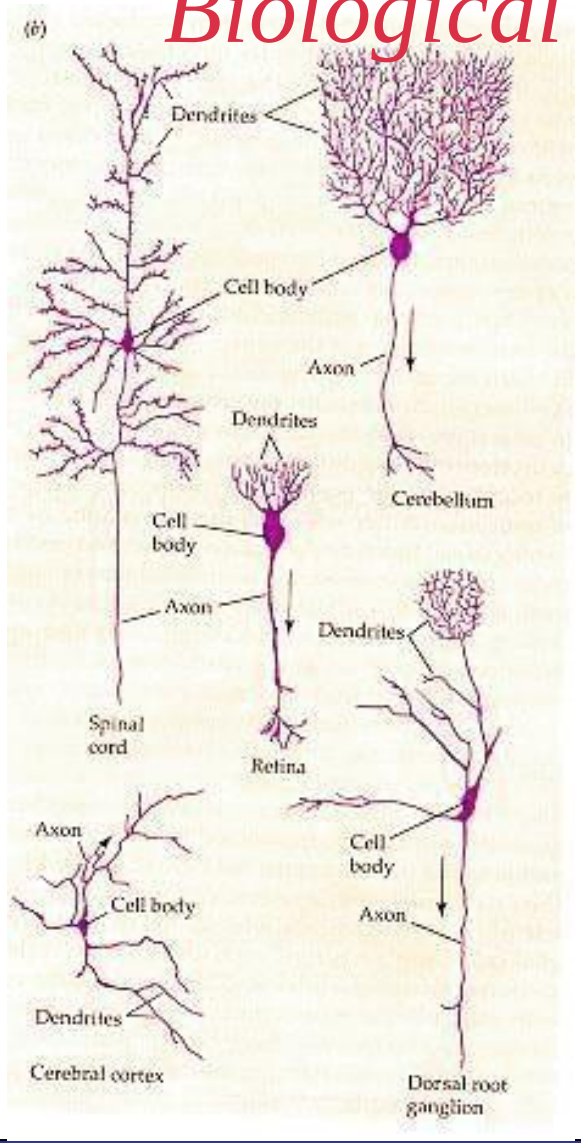
- For example, when shown millions of images taken from the Internet, a computer vision system can identify a large cluster of similar images which an English speaker would call “cats.”
- In **reinforcement learning** the agent learns from a series of reinforcements: rewards learning and punishments.
- For example, at the end of a chess game the agent is told that it has won (a reward) or lost (a punishment).
- It is up to the agent to decide which of the actions prior to the reinforcement were most responsible for it, and to alter its actions to aim towards more rewards in the future.

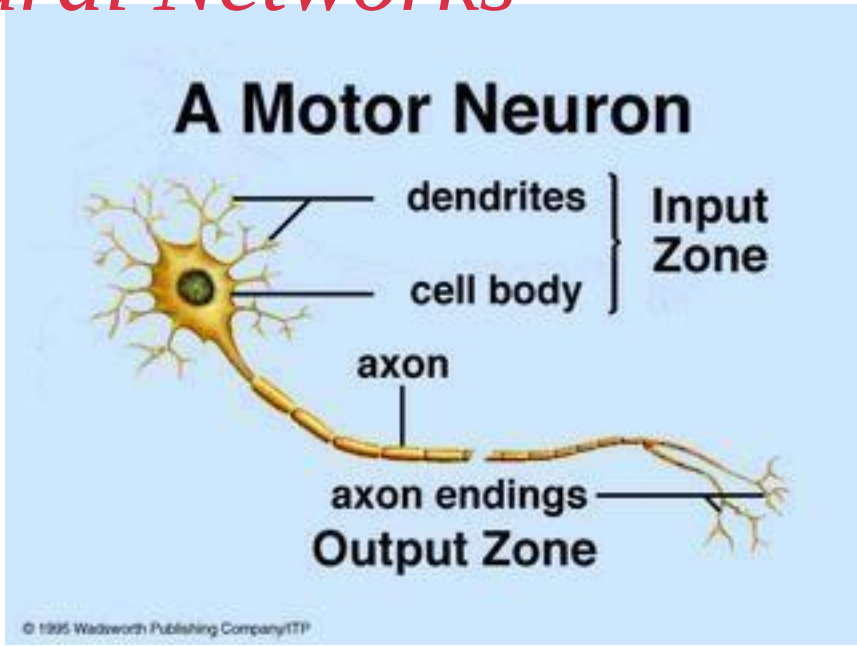
Definition of ANN

"Information processing system consisting of a large number of highly interconnected processing elements (artificial neurons) in an architecture inspired by the structure of the cerebral cortex of the brain"

(Tsoukalas & Uhrig, 1997).

Biological Neural Networks

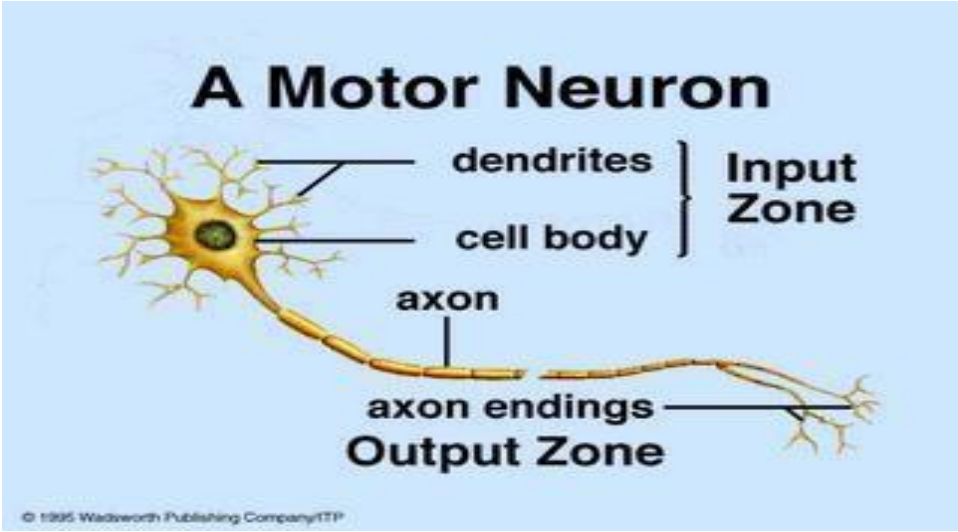




Biological neuron

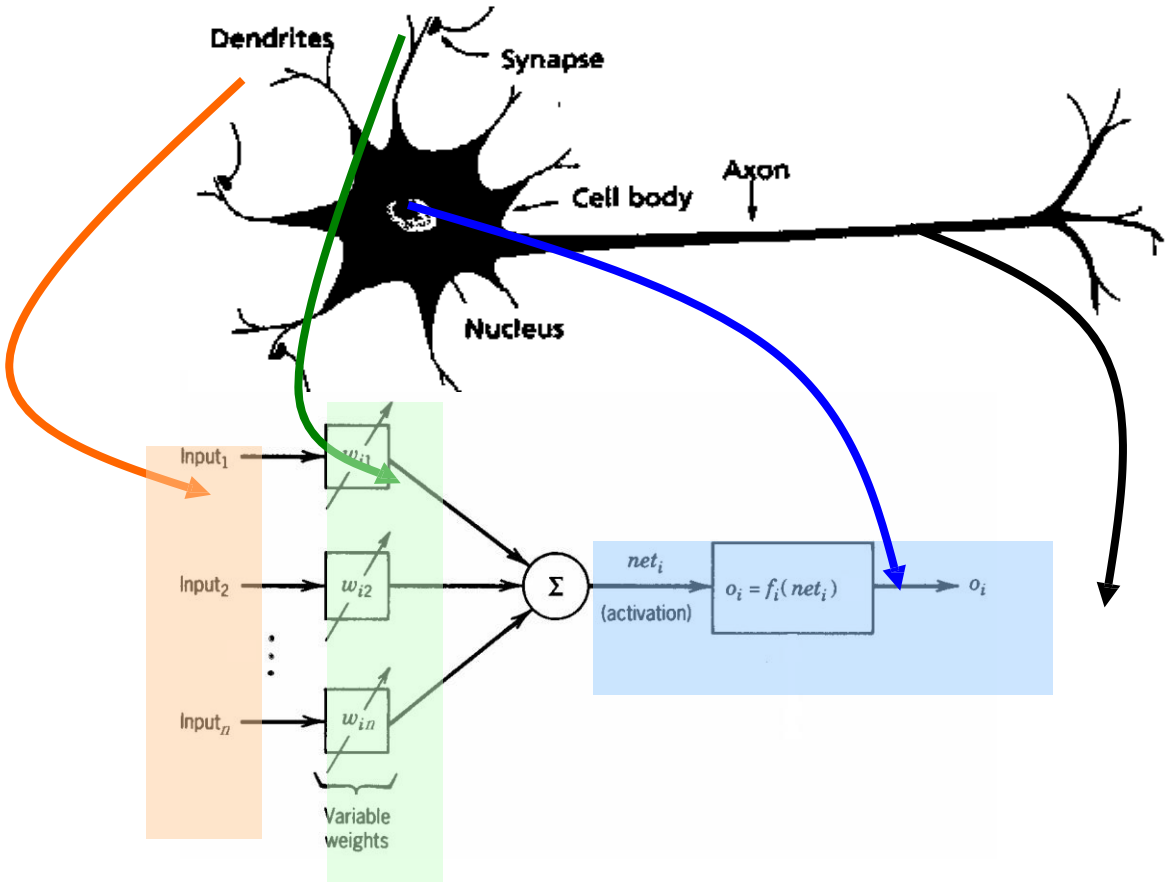
Biological Neural Networks

- **Soma or cell body-where the cell nucleus is located**
- **Dendrites-where the nerve is connected to the cell body**
- **Axon-single long connection extending from the cell body and carrying signals from the neuron**



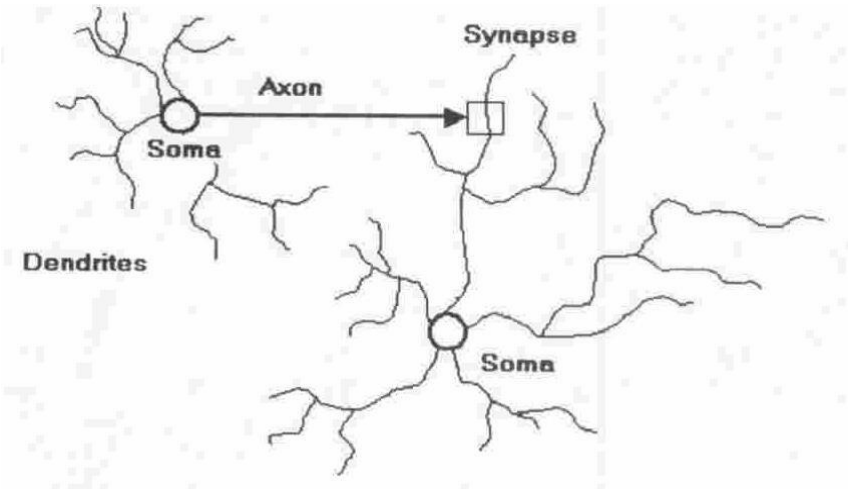
Artificial Neurons

A physical neuron

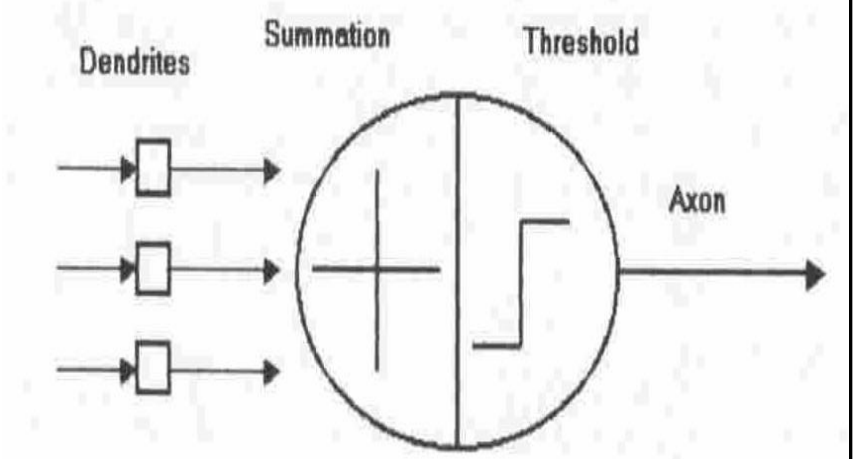


An artificial neuron

Artificial Neuron

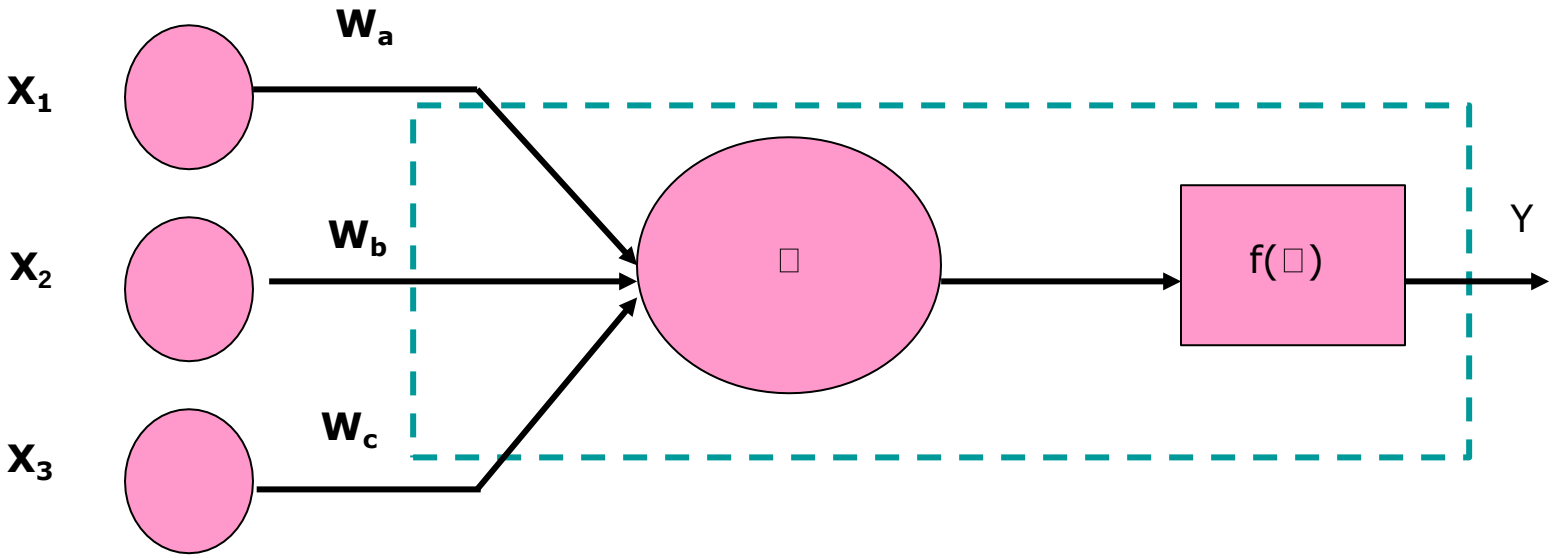


Four basic components of a human biological neuron



The components of a basic artificial neuron

Model Of A Neuron



Input units

Connection weights

Summing function

computation

(dendrite)

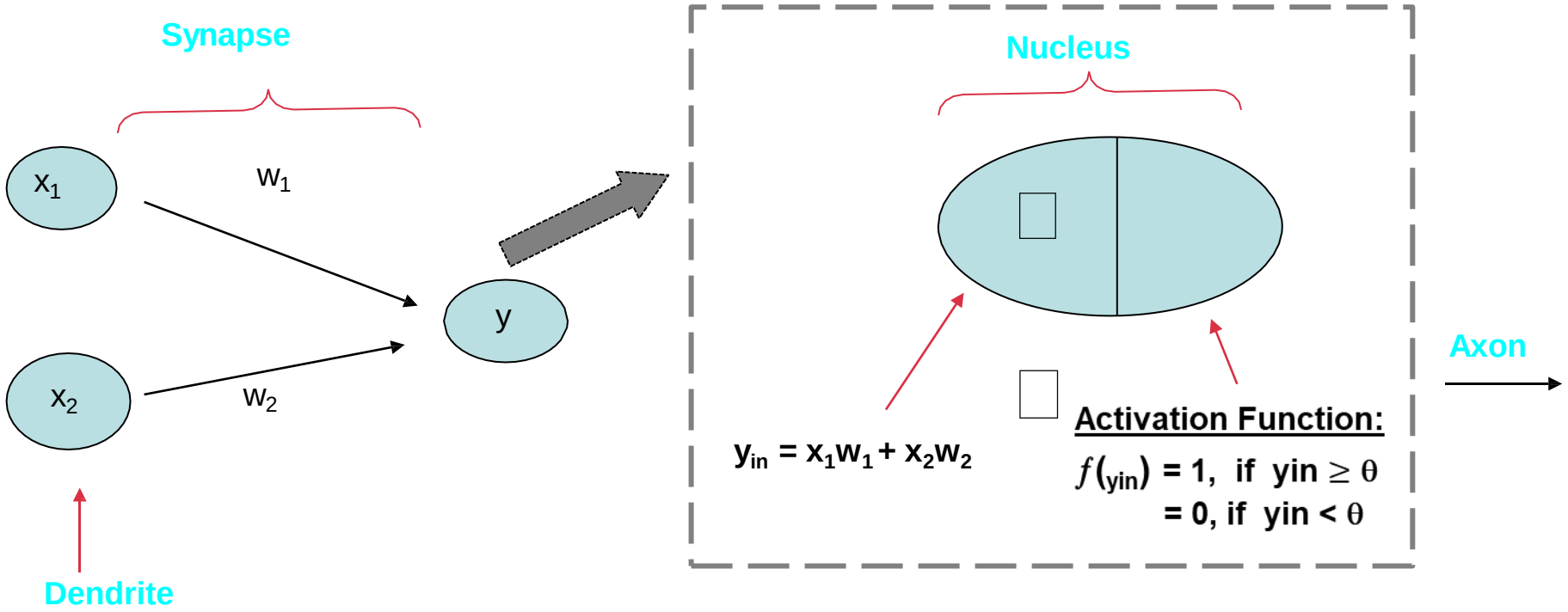
(synapse)

(soma)

(axon)

- Neural networks are configured for a specific application, such as pattern recognition or data classification, through a learning process.
- In a biological system, learning involves adjustments to the synaptic connections between neurons.
- Same for artificial neural networks (ANNs)

Artificial Neural Network



-A neuron receives input, determines the strength or the weight of the input, calculates the total weighted input, and compares the total weighted input with a threshold value (□)

-The value is in the range of 0 and 1

- If the total weighted input is greater than or equal the threshold value, the neuron will produce the output, and if the total weighted input less than the threshold value, no output will be produced

Bias

- Bias is an extra parameter added to neurons, allowing the network to have better flexibility in fitting data.
- It helps in shifting the activation function either left or right, improving the model's ability to learn patterns in data. Mathematically, it is added to the weighted sum of inputs before applying the activation function.
- **Example**: In the equation $y=f(w \cdot x+b)$. b is the bias, where www is the weight, x is the input, and f is the activation function.

Weight

- Weights are the parameters within the neural network that transform input data within a layer.
- Each input to a neuron is multiplied by a corresponding weight, which signifies the importance or strength of that input in determining the output.
- **Example**: For a neuron with inputs x_1, x_2, x_3, the output could be $y=f(w_1x_1+w_2x_2+w_3x_3+b)$, where w_1, w_2, w_3 are the weights.

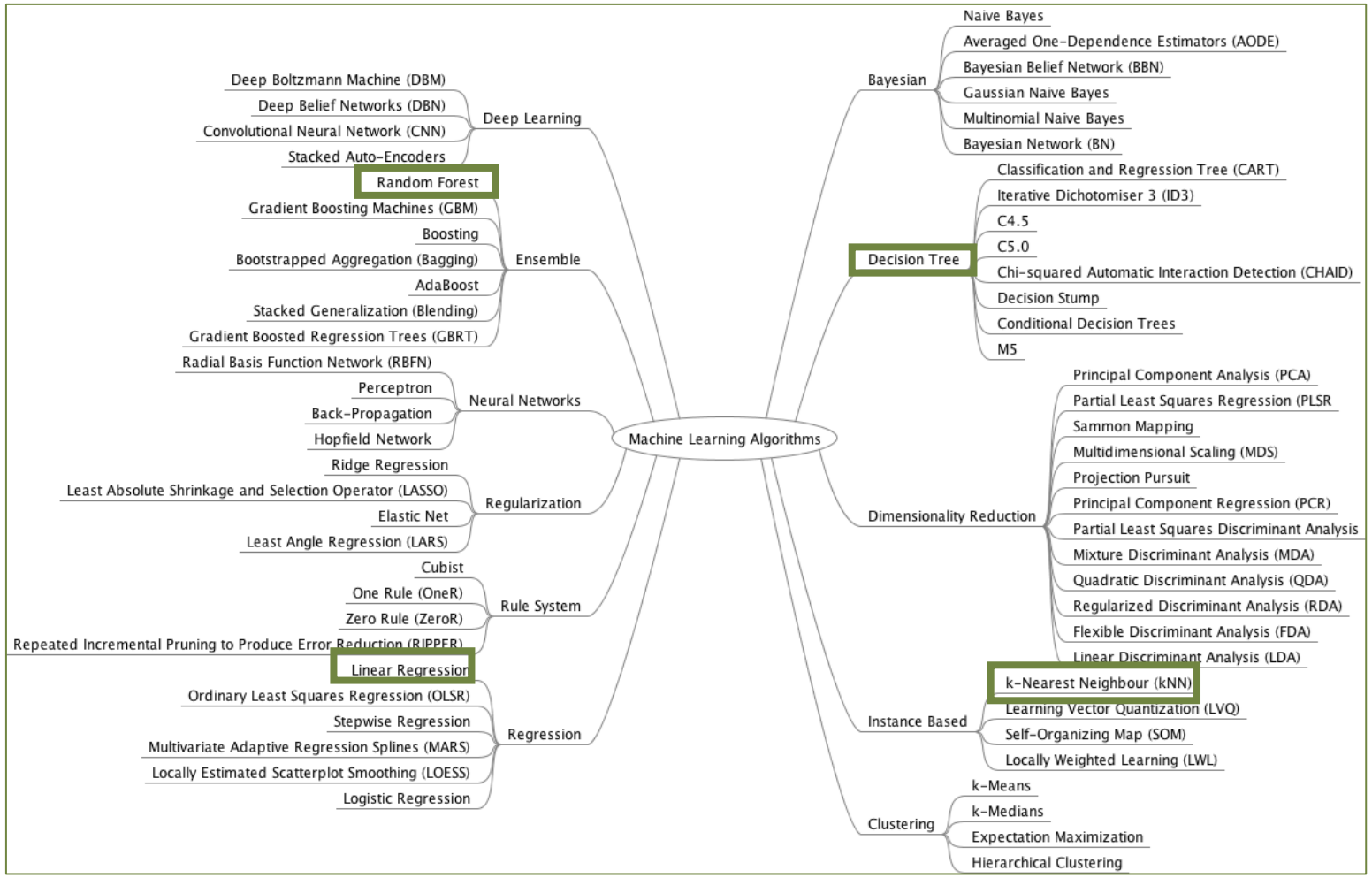
Threshold

- The threshold defines a boundary for the output activation. When the combined weighted inputs exceed the threshold, the neuron becomes activated, otherwise, it remains inactive.
- This is closely related to the activation function. It decides whether the output should be triggered or suppressed.
- In a step activation function, if the input exceeds a certain threshold, the neuron outputs 1; otherwise, it outputs 0.

Learning Rate (α)

- The learning rate is a hyperparameter that controls the step size at which the model's weights are updated during training.
- A high learning rate can make the model learn quickly, but might overshoot optimal solutions, while a low learning rate leads to slow convergence and can get stuck in local minima.

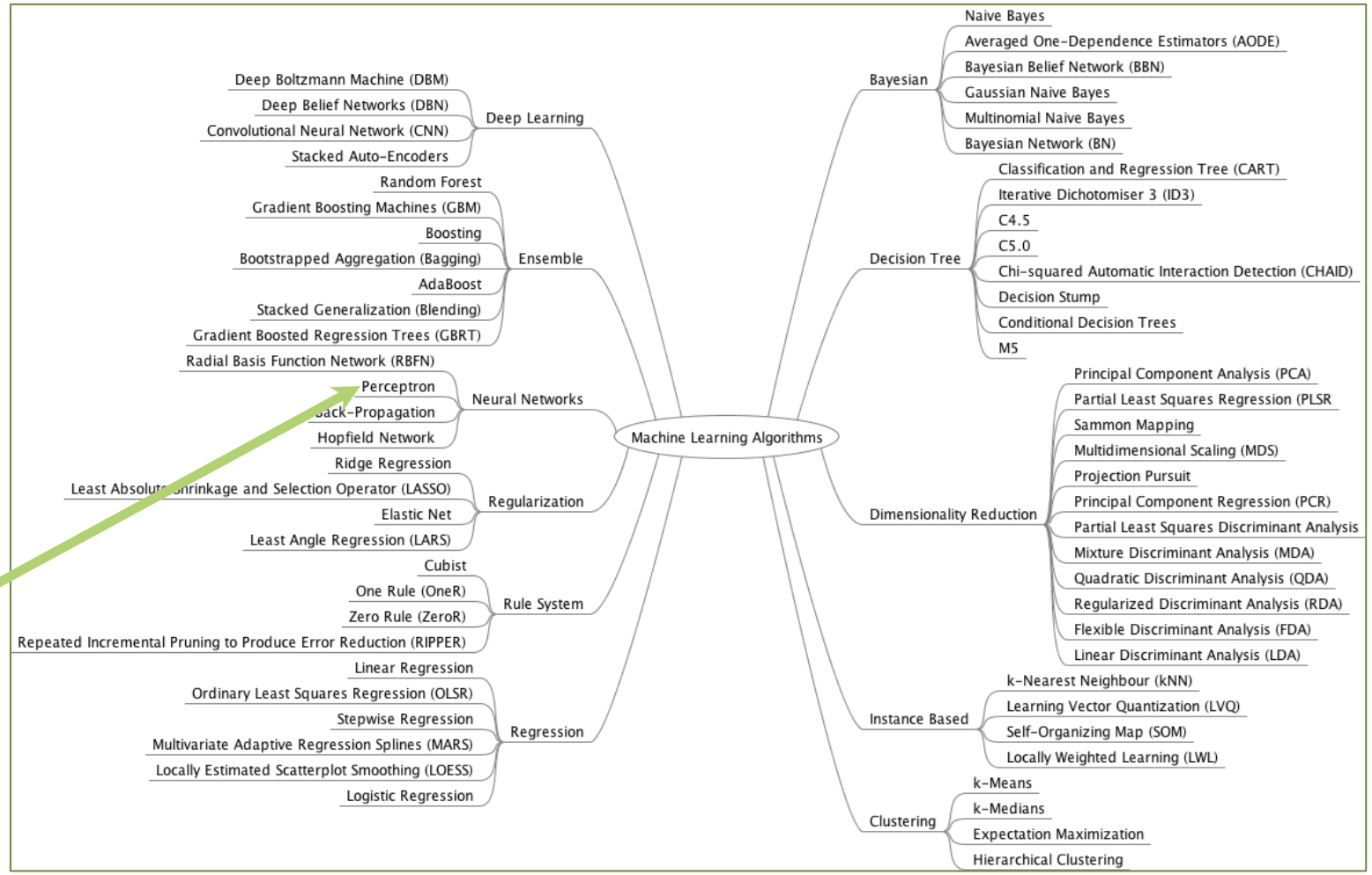
Types of AI Models



[Image of a network of Machine Learning Algorithm]. Retrieved October 9, 2020, from https://towardsdatascience.com/which-machine-learning-model-to-use-db5fdf37f3dd

Types of AI Models

Perceptron

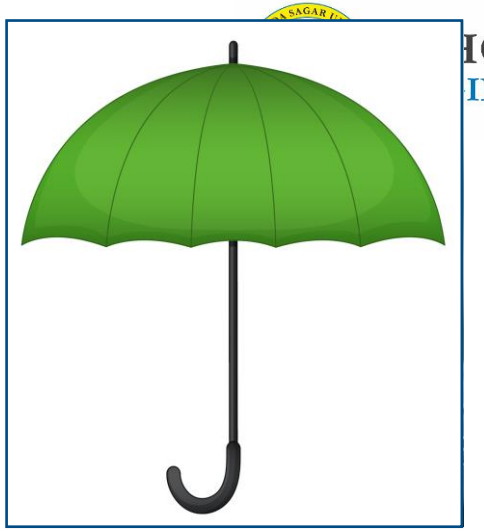


[Image of a network of Machine Learning Algorithm]. Retrieved October 9, 2020, from https://towardsdatascience.com/which-machine-learning-model-to-use-db5fdf37f3dd

How does AI make a Decision?

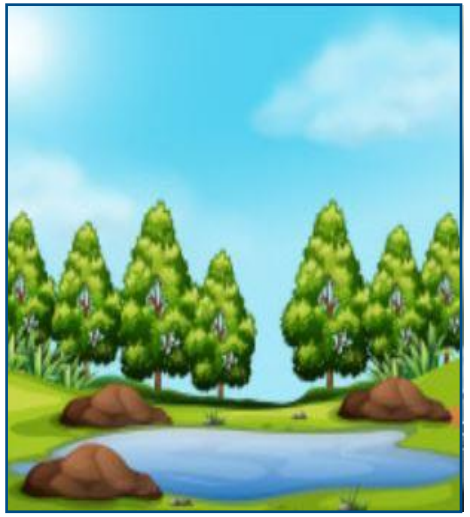
If you want to go to the park, how do you make your decision?

Do I have a jacket?



Do I have an umbrella?

Is it raining now?



What is the weather forecast for later?

brgfx[Image].Retrieved October 9, 2020, from https://www.freepik.com/vectors/people
brgfx[Image].Retrieved October 9, 2020, from https://www.freepik.com/vectors/green
brgfx[Image].Retrieved October 9, 2020, from https://www.freepik.com/vectors/background

How does AI make a Decision?

If you want to go to the park, how do you make your decision?

3rd

Do I have a jacket?

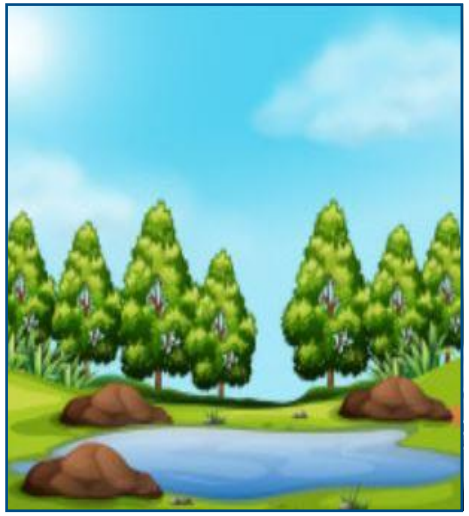
4th

Do I have an umbrella?

1st

Is it raining now?

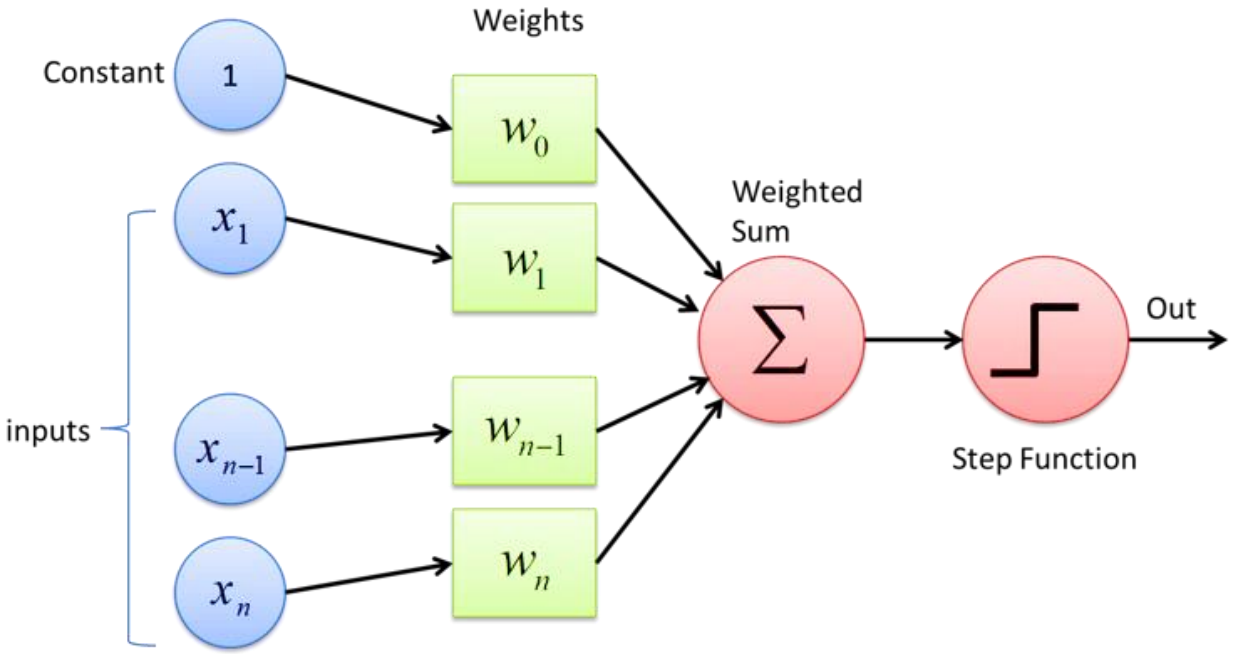
2nd



What is the weather forecast for later?

brgfx[Image].Retrieved October 9, 2020, from https://www.freepik.com/vectors/people
brgfx[Image].Retrieved October 9, 2020, from https://www.freepik.com/vectors/green
brgfx[Image].Retrieved October 9, 2020, from https://www.freepik.com/vectors/background

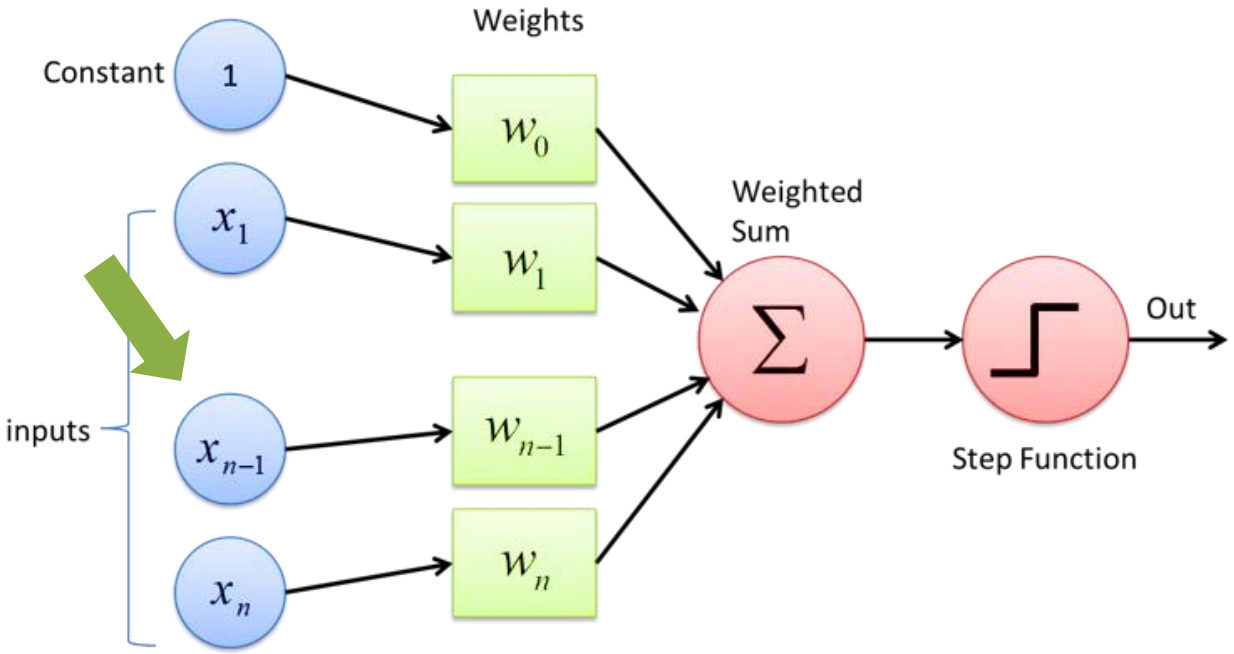
What is a Perceptron?



[Image of perceptron]. Retrieved October 9, 2020, from https://deepai.org/machine-learning-glossary-and-terms/perceptron

What is a Perceptron?

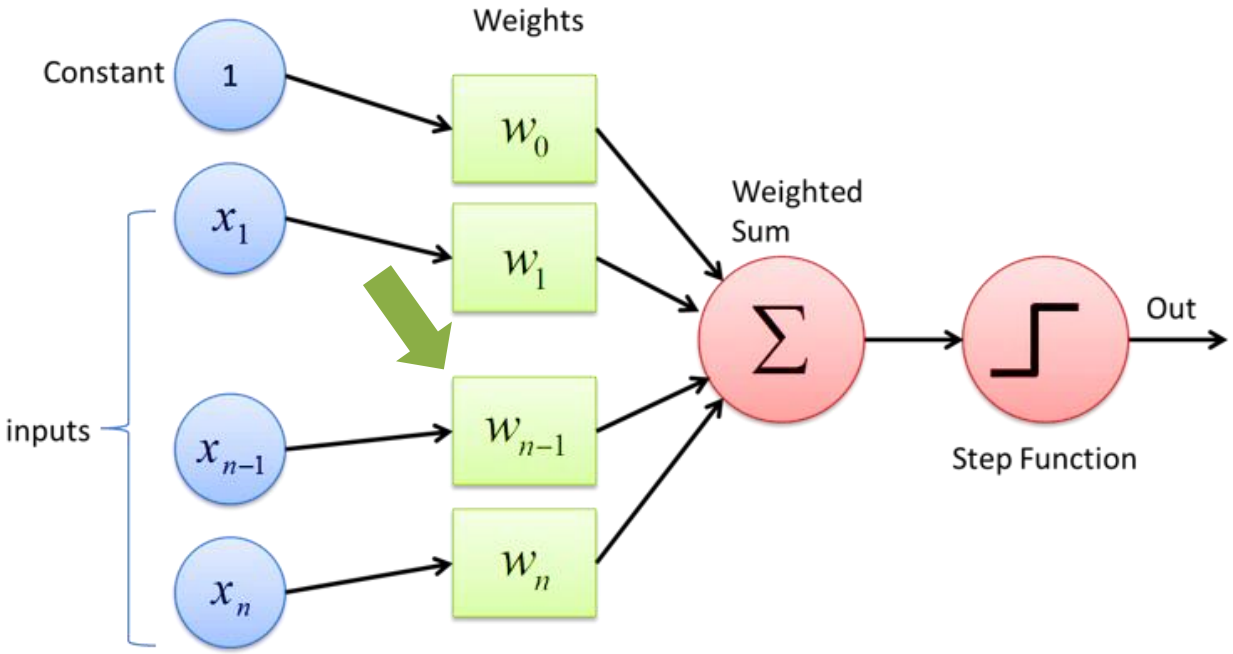
Add all the inputs (factors/conditions/ features) in



[Image of perceptron]. Retrieved October 9, 2020, from https://deepai.org/machine-learning-glossary-and-terms/perceptron

What is a Perceptron?

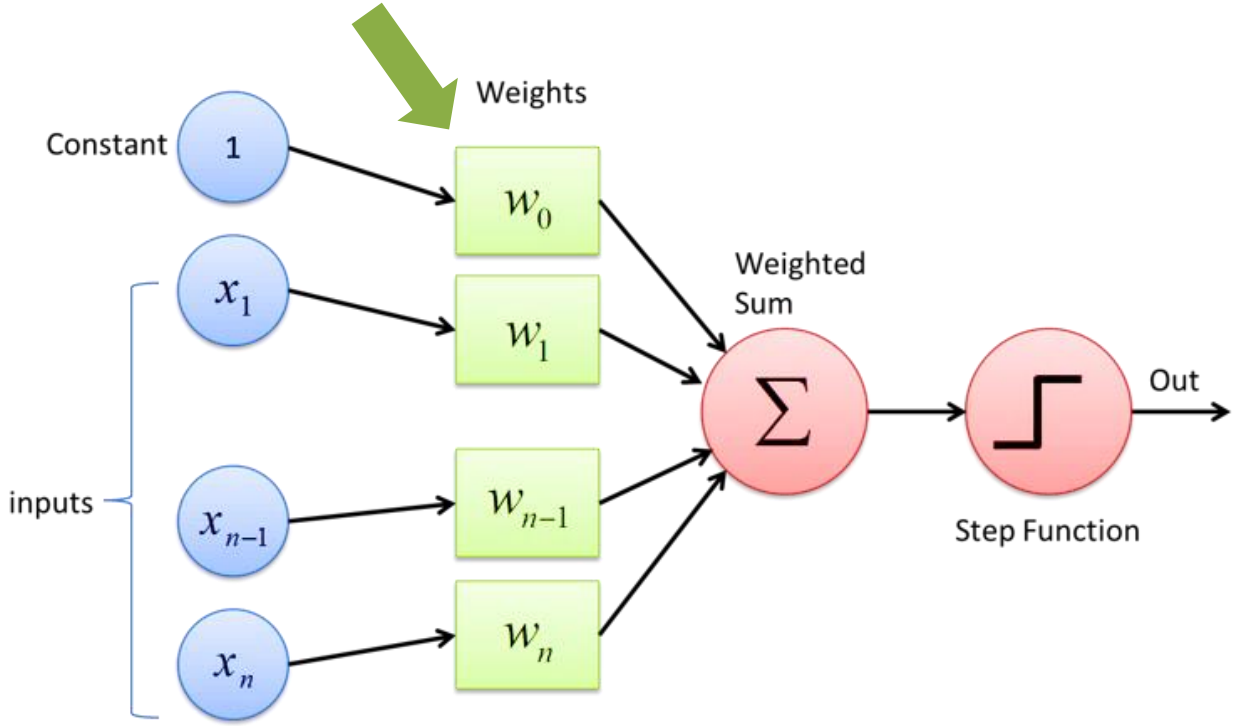
Introduce weights for the importance of the respective inputs



[Image of perceptron]. Retrieved October 9, 2020, from https://deepai.org/machine-learning-glossary-and-terms/perceptron

What is a Perceptron?

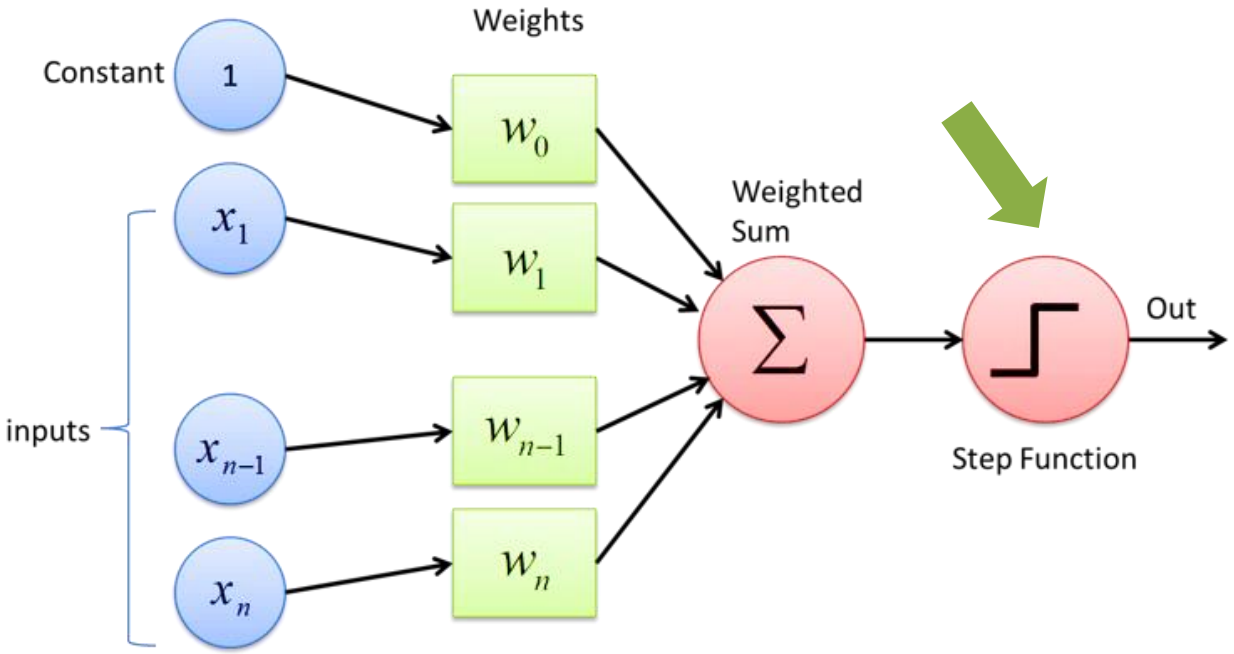
Introduce a ‘bias’ to get a ‘Yes’ or ‘No’ decision



[Image of perceptron]. Retrieved October 9, 2020, from https://deepai.org/machine-learning-glossary-and-terms/perceptron

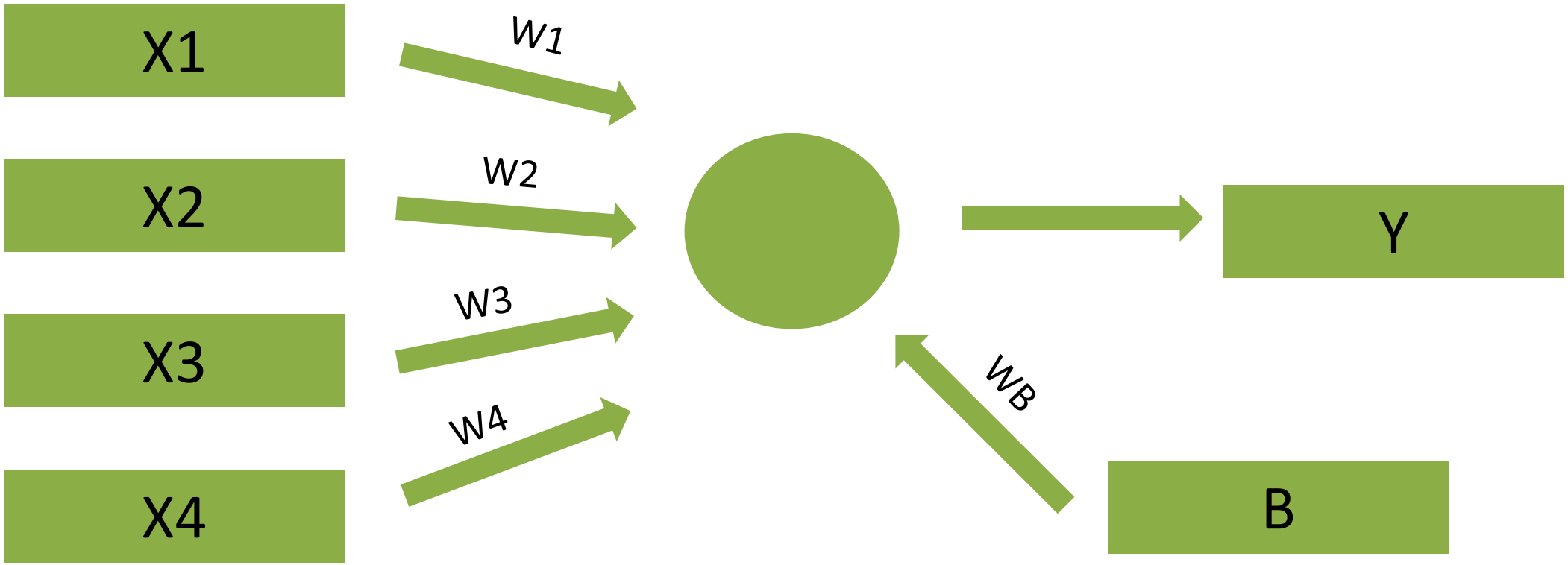
What is a Perceptron?

What is the outcome?
Positive value = Yes
Negative value = No

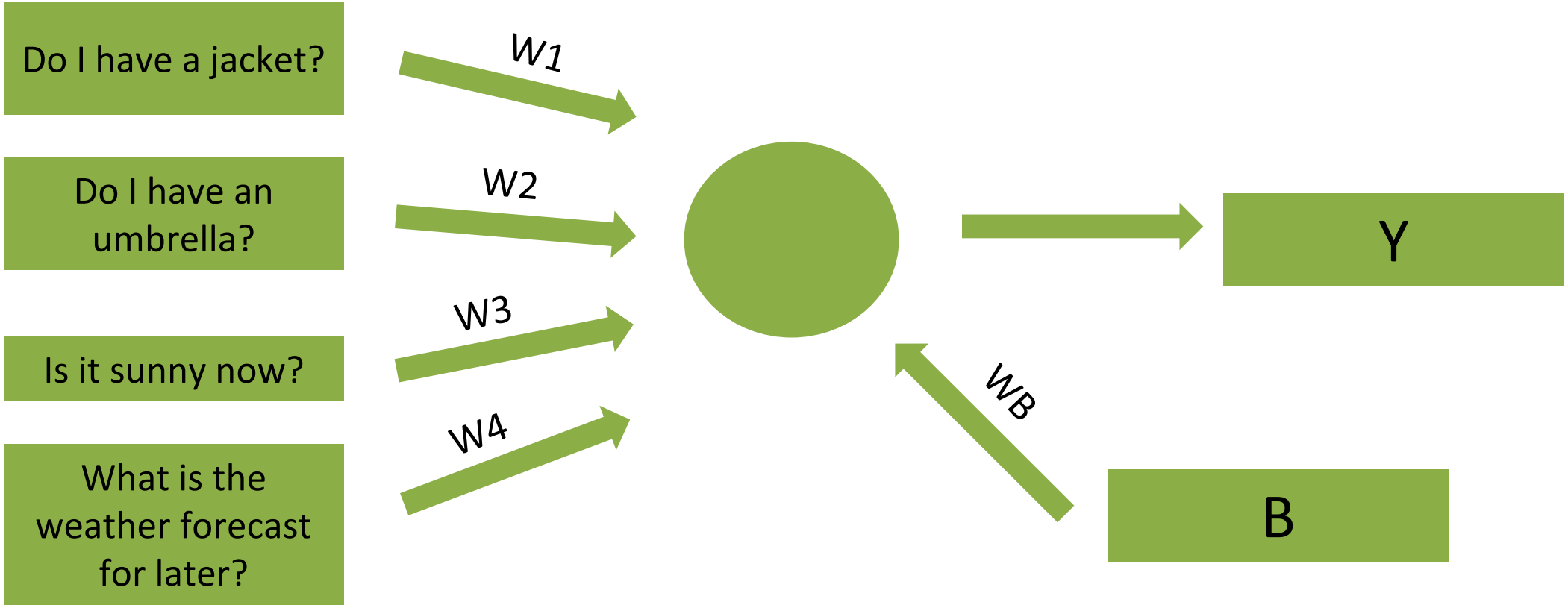


[Image of perceptron]. Retrieved October 9, 2020, from https://deepai.org/machine-learning-glossary-and-terms/perceptron

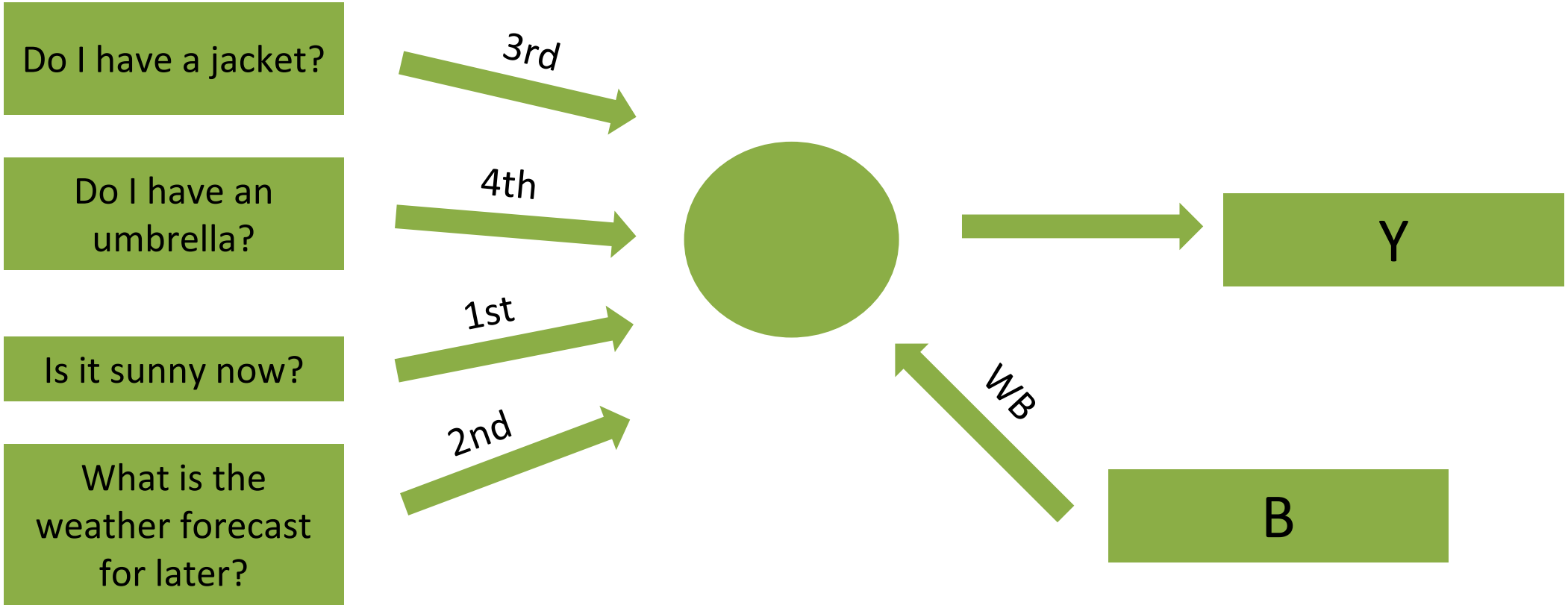
Perceptron Example (go to the park)



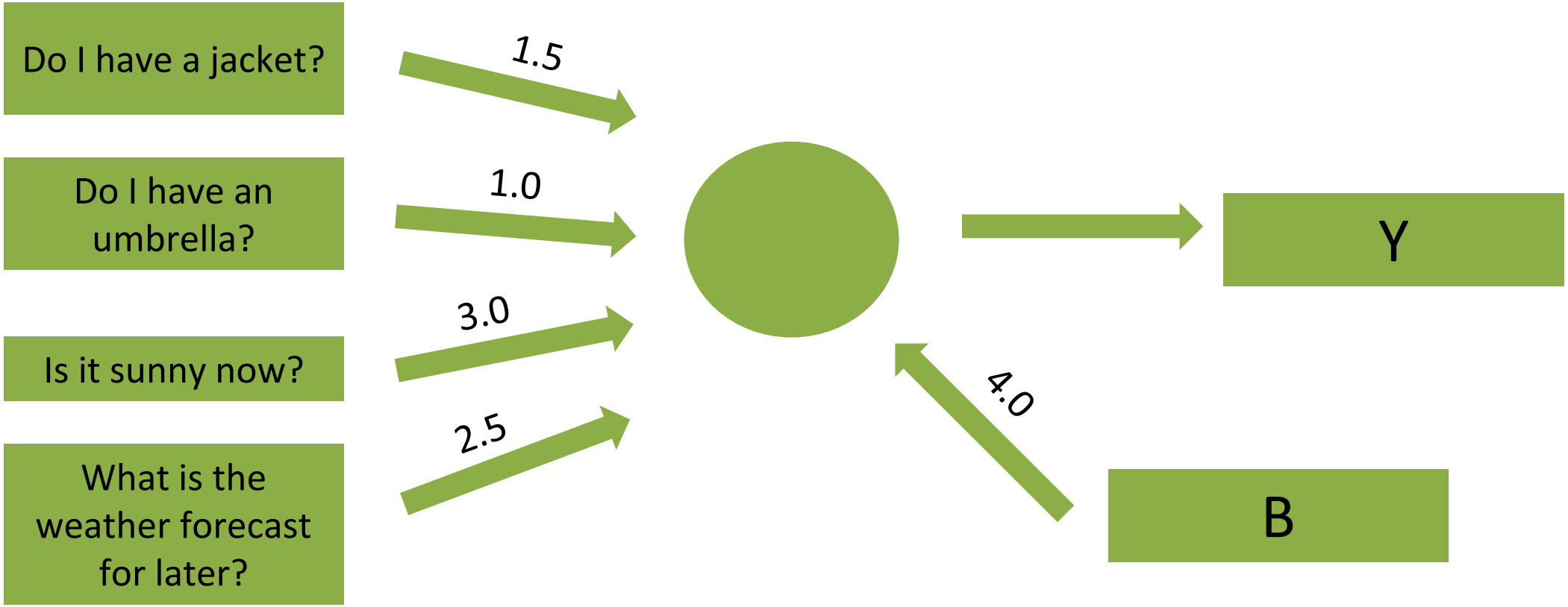
Perceptron Example (go to the park)



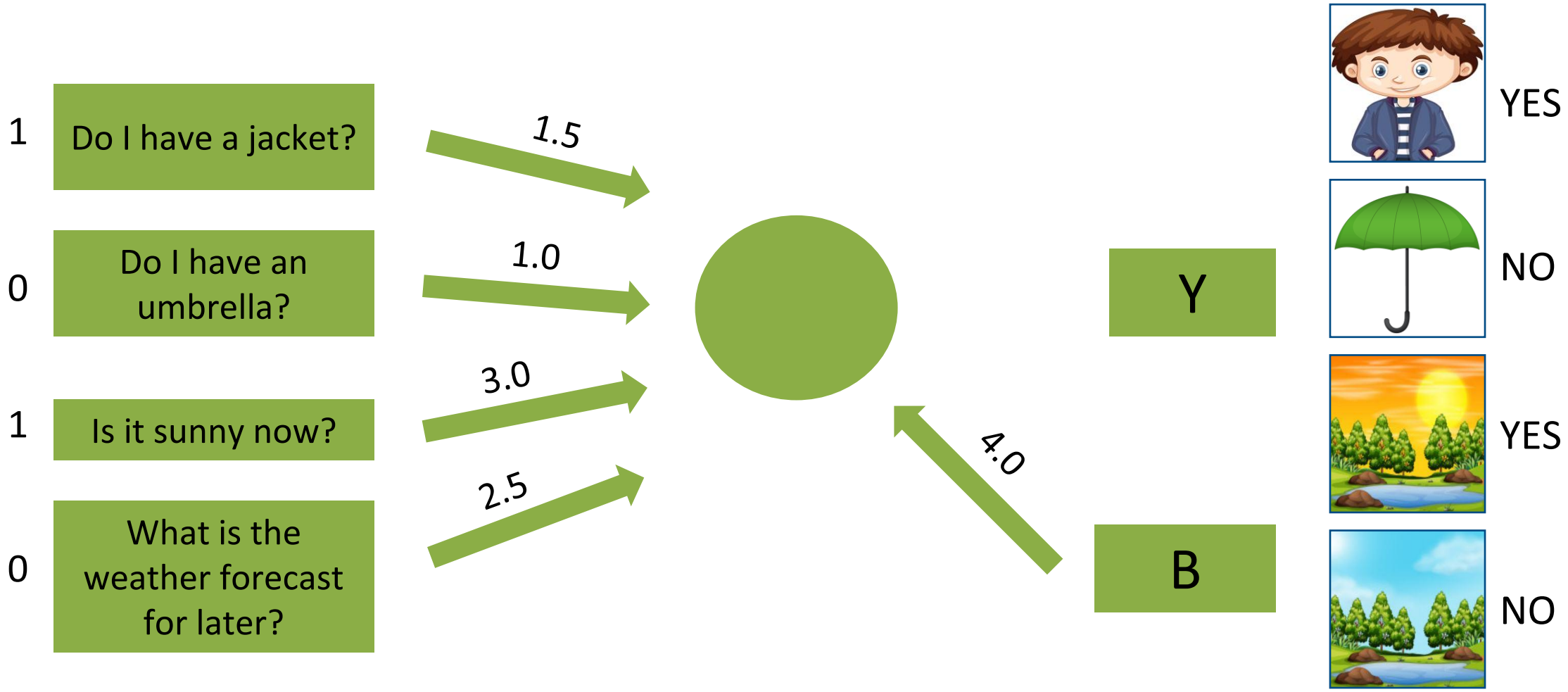
Perceptron Example (go to the park)



Perceptron Example (go to the park)



Perceptron Example (go to the park)



Scenario 1

Perceptron Example (go to the park)

1 Do I have a jacket?

0 Do I have an umbrella?

1 Is it sunny now?

0 What is the weather forecast for later?

1.5

1.0

3.0

2.5

4.0

Output = (1 * 1.5) + (0 * 1.0) + (1 * 3.0) + (0 * 2.5) – (1 * 4.0)

Output = 0.5

B

YES

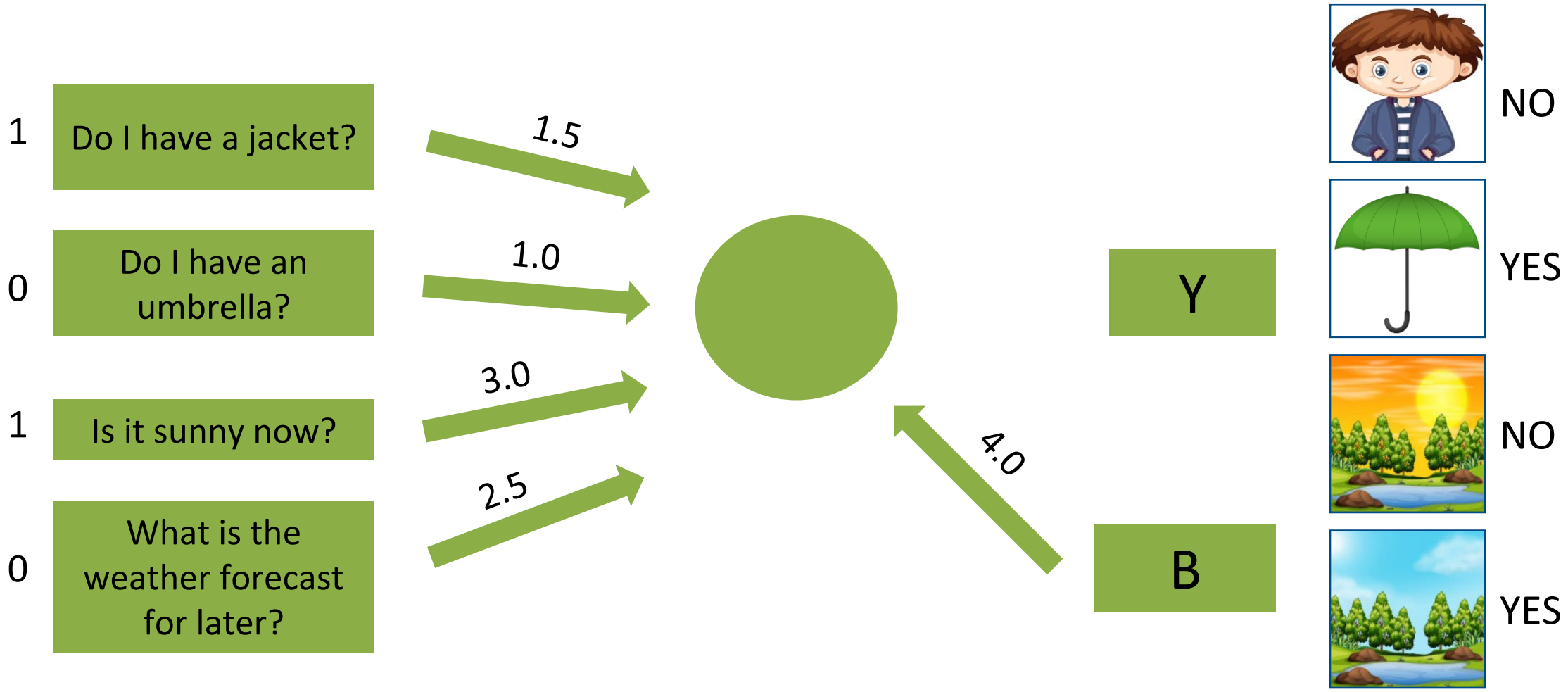
NO

YES

NO

Scenario 1

Perceptron Example (go to the park)



Scenario 2

Perceptron Example (go to the park)

1 Do I have a jacket? — 1.5

0 Do I have an umbrella? — 1.0

1 Is it sunny now? — 3.0

0 What is the weather forecast for later? — 2.5

Output = (0 * 1.5) + (1 * 1.0) + (0 * 3.0) + (1 * 2.5) – (1 * 4.0)

Output = - 0.5

Y

4.0

B

NO

YES

NO

YES

Scenario 2

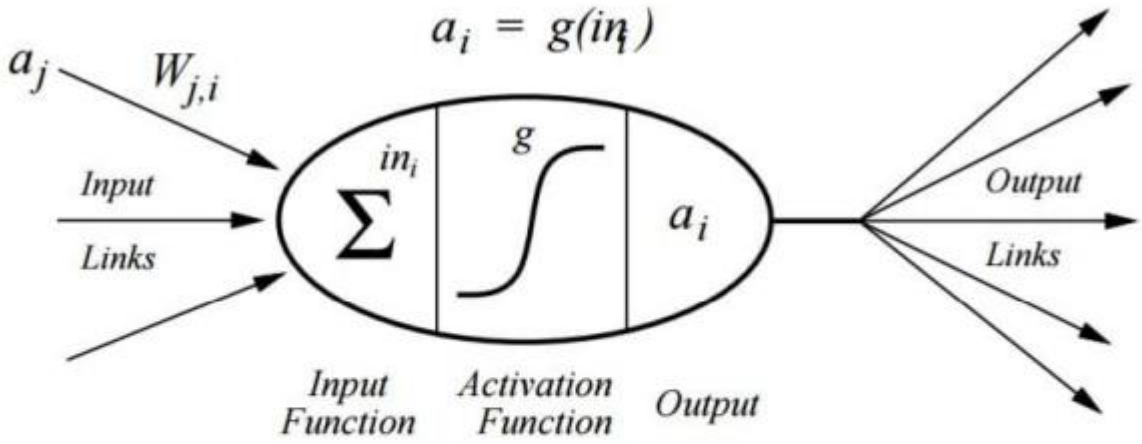
Group Activity

- Should I play basketball today?
- Should I study for my Math test?

Apply the learnings on perceptron for these examples

Sigmoid Activation Function

- The diagram given here shows a Perceptron with sigmoid activation function. Sigmoid is one of the most popular activation functions.



$$a_i = g(\sum_j W_{j,i} a_j)$$

Sigmoid Activation Function

- A Sigmoid Function is a mathematical function with a Sigmoid Curve ("S" Curve). It is a special case of the logistic function and is defined by the function given below:

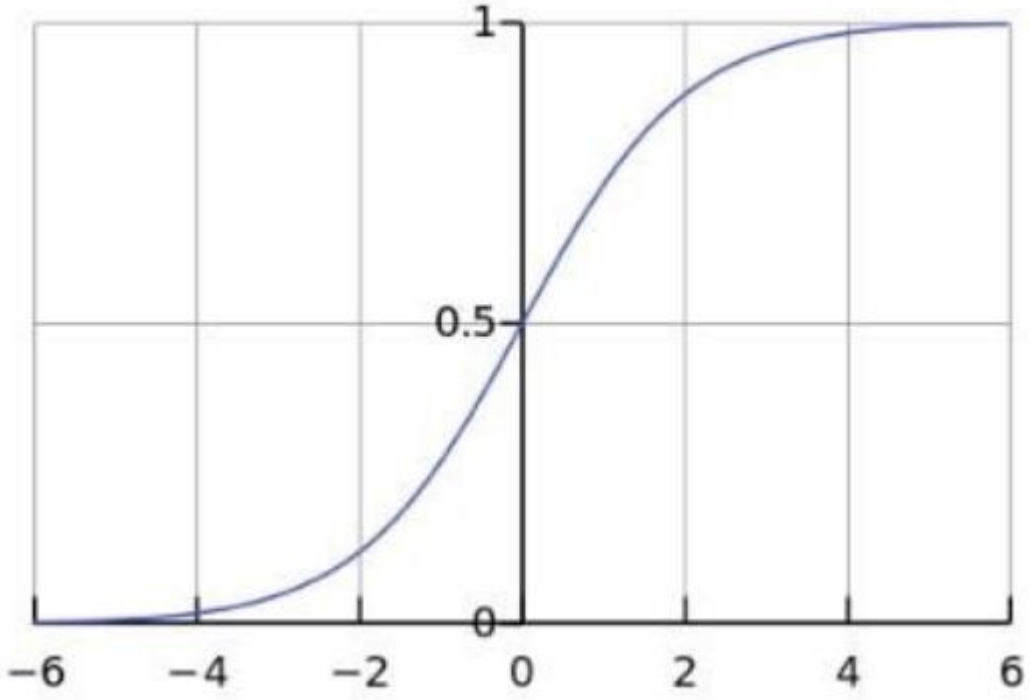
$$\phi_{logistic}(z) = \frac{1}{1+e^{-z}}$$

- Here, value of z is:

$$z = w_0 x_0 + w_1 x_1 + \cdots + w_m x_m = \sum_{i=0}^{m} w_i x_i = w^T x$$

Sigmoid Activation Function

- The curve of the Sigmoid function called “S Curve” is shown here.



Sigmoid Activation Function

- This is called a logistic sigmoid and leads to a probability of the value between 0 and 1.
- This is useful as an activation function when one is interested in probability mapping rather than precise values of input parameter t.
- The sigmoid output is close to zero for highly negative input.
- This can be a problem in neural network training and can lead to slow learning and the model getting trapped in local minima during training.
- Hence, hyperbolic tangent is more preferable as an activation function in hidden layers of a neural network.

OR Gate

- $w_1 = 0.5$
- $w_2 = 0.5$
- $b = -0.5$
- $\alpha = 1$
- $z = \sum x_i \, w_i + b$
- $\hat{y} = f(z) = \begin{cases} 1 & if \; z > 0 \\ 0 & else \end{cases}$
- $e = y - \hat{y}$
- $w_i = w_i + \alpha . e . x_i$
- $b = b + \alpha . e$

EPOCH	x_1	x_2	y	z	$\hat{y}$	e	w_1	w_2	b
Initial Values							0.5	0.5	-0.5
1	0	0	0	-0.5	0	0	0.5	0.5	-0.5
	0	1	1	0	0	1	0.5	1.5	0.5
	1	0	1	1	1	0	0.5	1.5	0.5
	1	1	1	2.5	1	0	0.5	1.5	0.5
2	0	0	0	0.5	1	-1	0.5	1.5	-0.5
	0	1	1	1	1	0	0.5	1.5	-0.5
	1	0	1	0	0	1	1.5	1.5	0.5
	1	1	1	3.5	1	0	1.5	1.5	0.5
3	0	0	0	0.5	1	-1	1.5	1.5	-0.5
	0	1	1	1	1	0	1.5	1.5	-0.5
	1	0	1	1	1	0	1.5	1.5	-0.5
	1	1	1	2.5	1	0	1.5	1.5	-0.5
4	0	0	0	-0.5	0	0	1.5	1.5	-0.5
	0	1	1	1	1	0	1.5	1.5	-0.5
	1	0	1	1	1	0	1.5	1.5	-0.5
	1	1	1	2.5	1	0	1.5	1.5	-0.5

AND Gate

- $w_1 = 0.5$
- $w_2 = 0.5$
- b=0.5
- α=1
- $z = \sum x_i \, w_i + b$
- $\hat{y}$ =f(z)=$\begin{cases} 1 & if\ z > 0 \\ 0 & else \end{cases}$
- e=y- $\hat{y}$
- $w_i = w_i + \alpha . e . x_i$
- b=b+ α.e

EPOCH	x_1	x_2	y	z	$\hat{y}$	e	w_1	w_2	b
Initial Values							0.5	0.5	0.5
1	0	0	0	0.5	1	-1	0.5	0.5	-0.5
	0	1	0	0	0	0	0.5	0.5	-0.5
	1	0	0	0	0	0	0.5	0.5	-0.5
	1	1	1	0.5	1	0	0.5	0.5	-0.5
2	0	0	0	-0.5	0	0	0.5	0.5	-0.5
	0	1	0	0	0	0	0.5	0.5	-0.5
	1	0	0	0	0	0	0.5	0.5	-0.5
	1	1	1	0.5	1	0	0.5	0.5	-0.5

AND Gate

- $w_1 = 0.5$
- $w_2 = 0.5$
- b=-1
- α=1
- $z = \sum x_i \, w_i + b$
- $\hat{y}$ =f(z)=$\begin{cases} 1 & if\ z > 0 \\ 0 & else \end{cases}$
- e=y- $\hat{y}$
- $w_i = w_i + \alpha . e . x_i$
- b=b+ α.e

EPOCH	x_1	x_2	y	z	$\hat{y}$	e	w_1	w_2	b	α
Initial Values							0.5	0.5	-1	1
1	0	0	0	-1	0	0	0.5	0.5	-1	1
	0	1	0	-0.5	0	0	0.5	0.5	-1	1
	1	0	0	-0.5	0	0	0.5	0.5	-1	1
	1	1	1	0	0	1	1.5	1.5	0	1
2	0	0	0	0	0	0	1.5	1.5	0	1
	0	1	0	1.5	1	-1	1.5	0.5	-1	1
	1	0	0	0.5	1	-1	0.5	0.5	-2	1
	1	1	1	-1	0	1	1.5	1.5	-1	1
3	0	0	0	-1	0	0	1.5	1.5	-1	1
	0	1	0	0.5	1	-1	1.5	0.5	-2	1
	1	0	0	-0.5	0	0	1.5	0.5	-2	1
	1	1	1	0	0	1	2.5	1.5	-1	1

EPOCH	x_1	x_2	y	z	$\hat{y}$	e	w_1	w_2	b	α
4	0	0	0	-1	0	0	2.5	1.5	-1	1
	0	1	0	0.5	1	-1	2.5	0.5	-2	1
	1	0	0	0.5	1	-1	1.5	0.5	-3	1
	1	1	1	-1	0	1	2.5	1.5	-2	1
5	0	0	0	-2	0	0	2.5	1.5	-2	1
	0	1	0	-0.5	0	0	2.5	1.5	-2	1
	1	0	0	0.5	1	-1	1.5	1.5	-3	1
	1	1	1	0	0	1	2.5	2.5	-2	1
6	0	0	0	-2	0	0	2.5	2.5	-2	1
	0	1	0	0.5	1	-1	2.5	1.5	-3	1
	1	0	0	-0.5	0	0	2.5	1.5	-3	1
	1	1	1	1	1	0	2.5	1.5	-3	1
7	0	0	0	-3	0	0	2.5	1.5	-3	1
	0	1	0	-1.5	0	0	2.5	1.5	-3	1
	1	0	0	-0.5	0	0	2.5	1.5	-3	1
	1	1	1	1	1	0	2.5	1.5	-3	1

Multilayer Perceptron

- Artificial neural network models, one of which is the perceptron take their inspiration from the brain.
- In engineering, our aim is not to understand the brain perse, but to build useful machines.
- We are interested in artificial neural networks because we believe that they may help us build better computer systems.
- The brain is an information processing device that has some incredible abilities and surpasses current engineering products in many domains—for example, vision, speech recognition, and learning.

Multilayer Perceptron

- The human brain is quite different from a computer. Whereas a computer generally has one processor, the brain is composed of a very large (10^{11}) number of processing units, namely, neurons, operating in parallel.
- Neurons in the brain have connections, called synapses, to around 10^4 other neurons, all operating in parallel.
- In a computer, the processor is active and the memory is separate and passive.
- In the brain, both the processing and memory are distributed together over the network; processing is done by the neurons, and the memory is in the synapses between the neurons.

Understanding the Brain

- Understanding an information system has three levels called levels of analysis:

1. **Computational theory** corresponds to the goal of computation and an abstract definition of the task.
2. **Representation and algorithm** is about how the input and the output are represented and about the specification of the algorithm for the transformation from the input to the output.
3. **Hardware implementation** is the actual physical realization of the system.

Understanding the Brain

- One example is sorting: The computational theory is to order a given set of elements.
- The representation may use integers, and the algorithm may be Quicksort.
- After compilation, the executable code for a particular processor sorting integers represented in binary is one hardware implementation.

Neural Networks as a Paradigm for Parallel Processing

- In parallel processing are mainly two paradigms for parallel processing:
- In Single Instruction Multiple Data (SIMD) machines, all processors execute the same instruction but on different pieces of data.
- In Multiple Instruction Multiple Data (MIMD) machines, different processors may execute different instructions on different data.
- SIMD machines are easier to program because there is only one program to write.
- Neural Instruction Multiple Data (NIMD) machines, where each processor corresponds to a neuron, local parameters correspond to its synaptic weights, and the whole structure is a neural network.

The Perceptron

- The perceptron is the basic processing element.
- It has inputs that may come from the environment or may be the outputs of other perceptrons.
- Associated with each input, x_j connection weight $\in$ R, j = 1, . . . , d, is a connection weight, or synaptic weight $w_j \in$ R, and the output, y, in the simplest case is a weighted sum of the inputs.

$$y = \sum_{j=1}^{d} w_j x_j + w_0$$

- w_0 is the intercept value to make the model more general, weight coming from an extra bias unit, x_0

The Perceptron

- $y = w^T x$
- where $w = [w0, w1, \ldots, wd]^T$ and $x = [1, x1, \ldots, xd]^T$ are augmented vectors to include also the bias weight and input.

Learning time

- In some applications, the input is temporal where we need to learn a temporal sequence.
- In others, the output may also change in time.
- **Sequence recognition**. This is the assignment of a given sequence to one of several classes. Speech recognition is one example where the input signal sequence is the spoken speech and the output is the code of the word spoken. That is, the input changes in time but the output does not.

Learning time

- In some applications, the input is temporal where we need to learn a temporal sequence.
- In others, the output may also change in time.
- **Sequence recognition**. This is the assignment of a given sequence to one of several classes. Speech recognition is one example where the input signal sequence is the spoken speech and the output is the code of the word spoken. That is, the input changes in time but the output does not.

Learning time

- **Sequence reproduction**. Here, after seeing part of a given sequence, the system should predict the rest. Time-series prediction is one example where the input is given but the output changes.
- **Temporal association**. This is the most general case where a particular output sequence is given as output after a specific input sequence. The input and output sequences may be different. Here both the input and the output change in time.

Time Delay Neural Networks

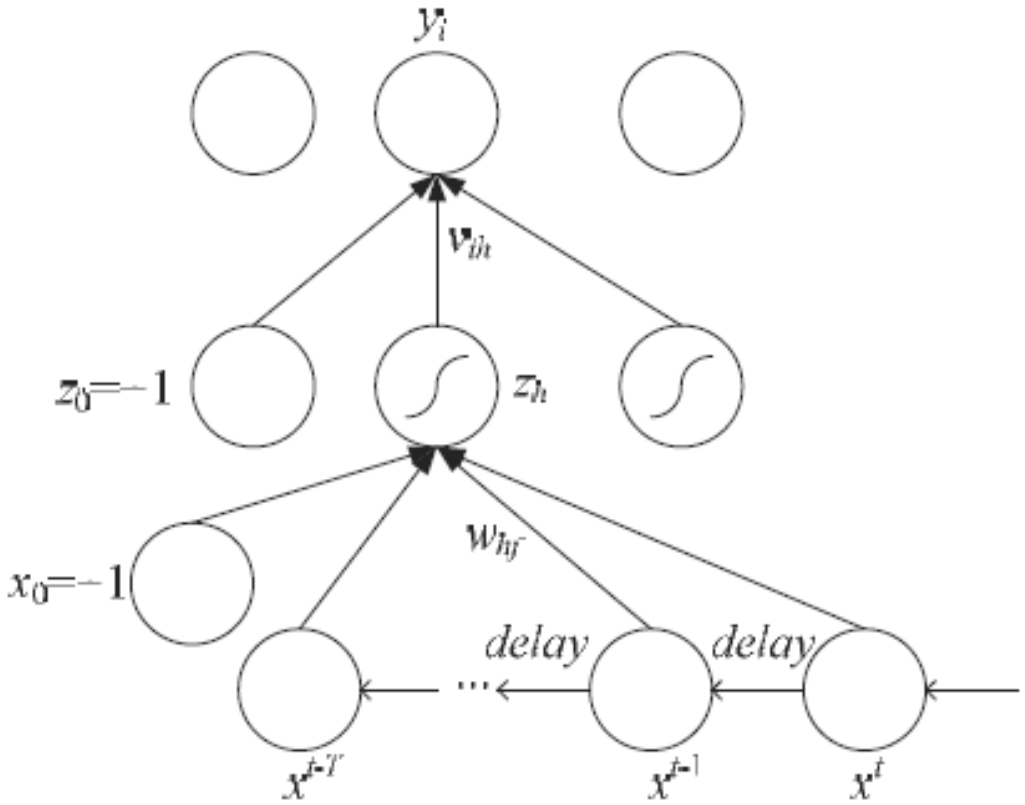
- The easiest way to recognize a temporal sequence is by converting it to a spatial sequence.
- In a time delay neural network previous inputs are delayed in time so as to synchronize with the final input, and all are fed together as input to the system Backpropagation can then be used to train the weights.
- To extract features local in time, one can have layers of structured connections and weight sharing to get translation invariance in time.
- The main restriction of this architecture is that the size of the time window we slide over the sequence should be fixed a priori.

Time Delay Neural Networks

- The easiest way to recognize a temporal sequence is by converting it to a spatial sequence.
- In a time delay neural network previous inputs are delayed in time so as to synchronize with the final input, and all are fed together as input to the system Backpropagation can then be used to train the weights.
- To extract features local in time, one can have layers of structured connections and weight sharing to get translation invariance in time.
- The main restriction of this architecture is that the size of the time window we slide over the sequence should be fixed a priori.

Time Delay Neural Networks

- In a recurrent network, additional to the feedforward connections, units have self-connections or connections to units in the previous layers.
- This recurrency acts as a short-term memory and lets the network remember what happened in the past.



Recurrent Networks

- In a recurrent network, additional to the feedforward connections, units have self-connections or connections to units in the previous layers.
- Most frequently, one uses a partially recurrent network where a limited number of recurrent connections are added to a multilayer perceptron
- This combines the advantage of the nonlinear approximation ability of a multilayer perceptron with the temporal representation ability of the recurrency, and such a network can be used to implement any of the three temporal association tasks.
- It is also possible to have hidden units in the recurrent backward connections, these being known as context units.

Recurrent Networks

- If the sequences have a small maximum length, then unfolding in time can be used to convert an arbitrary recurrent network to an equivalent feedforward network .
- A separate unit and connection is created for copies at different times. The resulting network can be trained with backpropagation with the additional requirement that all copies of each connection should remain identical.
- The solution, as in weight sharing, is to sum up the different weight changes in time and change the weight by the average. This is called backpropagation through time (Rumelhart, Hinton, and Willams 1986b).
- The problem with this approach is the memory requirement if the length of the sequence is large.
- Real time recurrent learning (Williams and Zipser 1989) is an algorithm learning for training recurrent networks without unfolding and has the advantage that it can use sequences of arbitrary length.

Deep Learning

- Deep learning is a subset of machine learning that uses neural networks with many layers (hence "deep") to analyze and model complex data patterns.
- It’s particularly effective in tasks like image and speech recognition, natural language processing, and game playing.

Key Concepts

- Neural Networks: Composed of layers of interconnected nodes (neurons) that process data and learn from it.
- Training: Involves feeding the model large amounts of labeled data so it can learn to make predictions.
- Backpropagation: A method for updating the weights of the network based on the error of predictions.
- Overfitting: When a model learns the training data too well, leading to poor performance on unseen data.
- Regularization: Techniques like dropout or L2 regularization are used to prevent overfitting.

Deep Learning

Applications

- Computer Vision: Image classification, object detection, facial recognition.
- Natural Language Processing: Language translation, sentiment analysis, chatbots.
- Reinforcement Learning: Training models through trial and error, often used in game AI.